INTERNAL · DO / SAY / SEE RUNBOOK · CODERABBIT FIELD ENGINEERING ENABLEMENT

Follow one finding: from Jira ticket to Deep Scan

A single feature, written by a coding agent, followed end to end: local review in the CLI and VS Code, the pull request review, Triage, Change Stack, QA Agent, the Security Deep Scan, and QA user journeys. The same two findings appear at every stop. Demo repository `coderabbit-demo/mmani-demo-prs`, PR #3, Jira story SC-10. Captured October 28, 2026, in light mode. Allow about 25 minutes plus questions.

> **OPEN WITH THIS** "One Jira ticket, one coding agent, one branch. Let's follow two things the agent got wrong — a security hole and a money bug — from the developer's laptop all the way to the security team's scan. Same findings, every stop."

How to read this runbook

Take one stage at a time, top to bottom. Each step has three parts.

COLUMN	WHAT IT HOLDS
Do	What to click, type, or open, with the deep link.
Say	The talk track. Deliver it aloud.
See	What the audience sees: the finding, check, diagram, or graph.

Callouts: `SECURITY` marks where security rides in. **WOW** is the moment to slow down. **Don't do this** flags a trap.

The dashboard

1	1	2	8
JIRA STORY · SC-10	CODING-AGENT PR · #3	TRACKED FINDINGS · F1 + F2	STOPS ON THE TRAIL

Index

Select any line to jump to that stage or step. Every step has an ↑ Index link back here.

Part I — The story: a ticket becomes a branch

Step 1 · The developer receives SC-10

Step 2 · The coding agent delivers a branch

Part II — The walkthrough

Stage 1 · Before the PR: CLI and VS Code

Step 3 · Review committed changes with the CodeRabbit CLI

Step 4 · Inspect the same findings in VS Code

Stage 2 · The pull request

Step 5 · Open PR #3 and read the summary

Step 6 · Walkthrough and sequence diagram

Step 7 · F1 The SQL injection finding
Step 8 · F2 The money bug: counted twice, never capped
Step 9 · F2 It breaks another repository
Step 10 · Pre-merge checks against Jira

Stage 3 · The reviewer: Triage and Change Stack

Step 11 · Triage: where does PR #3 sit?
Step 12 · Change Stack overview and layers
Step 13 · F1 The finding inside its layer
Step 14 · F1 Security Blast Radius
Step 15 · F1 F2 Security architecture impact

Stage 4 · QA Agent

Step 16 · Run the QA Agent on PR #3
Step 17 · F2 The money bug, proven in the running app
Step 18 · F1 The injection, exploited in the running app

Stage 5 · Application Security: Deep Scan

Step 19 · F1 Deep Scan ranks the same injection highest
Step 20 · F1 Reachability from the public route to the sink

Stage 6 · QA user journeys: same tests, two branches

Step 21 · Map the journey library with an agent
Step 22 · Baseline: all nine pass on main
Step 23 · F2 Same journeys on the agent branch: five fail
Step 24 · F2 Journey evidence: negative totals and a broken handoff

Part III — Reference

Appendix A — Deep-link index

Appendix B — Focused PRs for single-feature demos

Appendix C — Local setup for the CLI and VS Code demos

The two findings we follow

ID	FINDING	WHERE	WHY IT MATTERS
`F1 · SECURITY`	SQL injection in the discount-code lookup (CWE-89)	`services/orders/src/discounts.ts:20`	Shopper-controlled text is interpolated into SQL. A crafted code can select any discount row.
`F2 · BUG`	Discount stored as a negative-quantity line and counted twice; fixed discounts are not capped	`discounts.ts:29–30`, `:67–73`, `db.ts:17`	Totals are wrong (a $65.00 cart nets $52.00 instead of $58.50, and a $15.00 cart nets –$25.00). The downstream fulfillment service breaks.

The home base

ARTIFACT	LINK	ROLE
Jira story	SC-10 Discount codes at checkout	The task, with success criteria S1–S10
Pull request	mmani-demo-prs #3	Coding-agent delivery, branch `agent/discount-codes`
Consumer repo	mmani-demo-prs-consumer	Fulfillment and finance service broken by F2

Standards	coderabbit-standards/commerce	Order, discount, and data-access rules cited as Coding guidelines

PR #3 is open, unmerged, and labeled `Demo – Do not merge`, `coding-agent`, and `showcase: all-in-one`. The defects are intentional. The application is a demo storefront with synthetic data; never deploy it publicly.

Part I — The story: a ticket becomes a branch

STEP 1 The developer receives SC-10

↑ Index

DO

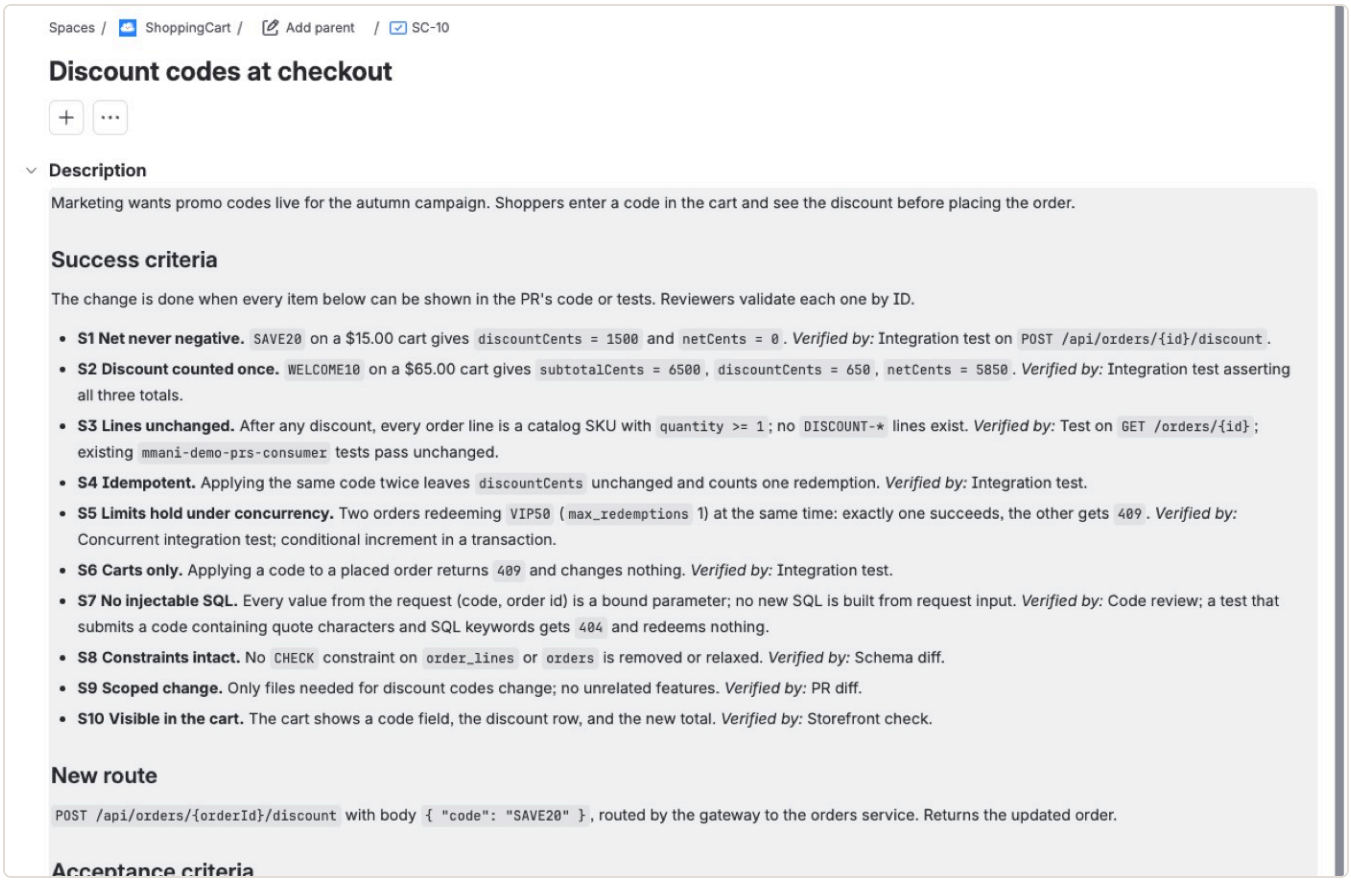
Open SC-10. Scroll through Success criteria S1–S10 and the Acceptance criteria.

SAY

"Marketing wants promo codes for the autumn campaign. The ticket is precise: discounts live on the order, the net is never negative, codes are bound parameters. Our developer is busy, so they hand it to a coding agent."

SEE

The story with S1 *Net never negative*, S3 *Lines unchanged*, S7 *No injectable SQL*. These are the exact criteria the agent will miss.



Jira SC-10 with numbered success criteria.

STEP 2 The coding agent delivers a branch

↑ Index

DO	SAY	SEE
Show the branch `agent/discount-codes` locally (`git log --oneline -3`). It was produced from the ticket by a coding agent.	*"The agent wrote the route, the database table, the cart field, and eight passing tests. Everything is green. Before I push this anywhere, I want a second pair of eyes."*	A green test run: `npm run check` passes 17 tests. Passing tests are not proof — two of them assert the wrong totals.

Part II — The walkthrough

Stage 1 · Before the PR: CLI and VS Code

Setup: `~/projects/mmani-demo-prs` on branch `agent/discount-codes`. The local branch `demo/agent-base` marks where the agent started, so the review shows only the agent's 13 files. Do not compare against `main`: it now holds the corrected reference implementation. Exact setup commands: Appendix C.

STEP 3 Review committed changes with the CodeRabbit CLI

↑ Index

DO

In a terminal, run:

```
cd ~/projects/mmani-demo-prs
coderabbit review --committed --base demo/agent-base
```

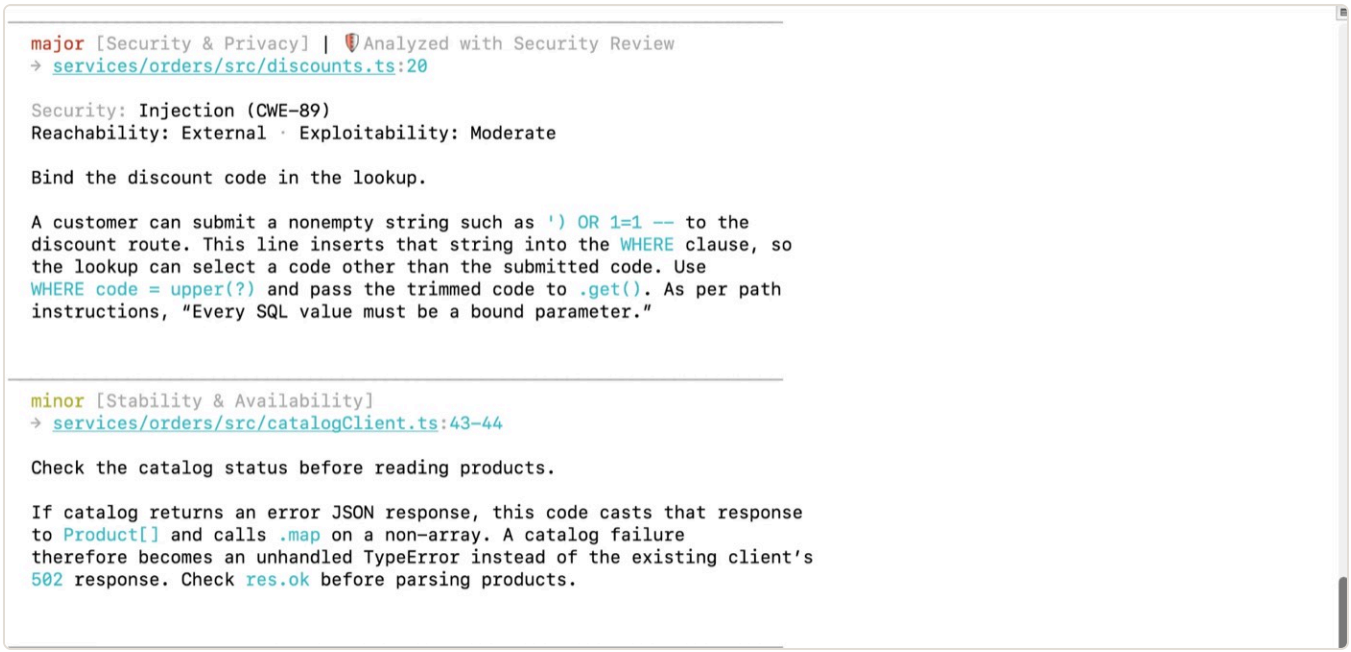
To list the stored findings again: `coderabbit review findings`

SAY

"Code exists, so review it now — before a PR, before a teammate spends time on it. The same engine that reviews PRs runs on my laptop."

SEE

F1 Injection (CWE-89) at `discounts.ts:20`: "Bind the discount code in the lookup." F2 "Limit the discount to the cart subtotal" (`discounts.ts:59`), and through the tests: "Do not assert a negative order total" and "Assert one deduction for the percent code" (5,200 should be 5,850).



CLI: F1, the SQL injection in the discount lookup.

```
minor [Security & Privacy] | Analyzed with Security Review
→ services/orders/src/discounts.ts:73

Security: Injection (CWE-89)
Reachability: External · Exploitability: Theoretical

Bind the order ID and amount in the discount update.

The UPDATE statement builds SQL with amountCents and order.id inside
a template literal. The path rule requires every SQL value to be a bound
parameter. The current values do not contain SQL syntax, so this is a
hardening fix, not an active injection. The route validates order IDs
against ID_PATTERN, and newOrderId() produces only alphanumeric
characters after the ord_ prefix. A future change to either rule could
make this statement injectable.


Use a prepared statement with bound parameters

-  db.exec(`UPDATE orders SET discount_cents = discount_cents + ${amountCents} WHERE id = '${order.id}'`);
+  db.prepare("UPDATE orders SET discount_cents = discount_cents + ? WHERE id = ?").run(amountCents, order.id);


major [Data Integrity & Integration]
→ services/orders/src/discounts.ts:59

Limit the discount to the cart subtotal.

For SAVE20 on a cart worth less than 2,000 cents, discountAmountCents
returns 2,000 without checking the subtotal. The subsequent writes can
make discountCents > subtotalCents and netCents negative. Reject or
cap an excessive discount before recording it. As per path instructions,
“0 <= discountCents <= subtotalCents; netCents is never negative.”
```

CLI: F2, the uncapped discount, plus the interpolated UPDATE at line 73.

```
mahesh@MacBook-Pro mmani-demo-prs % coderabbit review findings
Most recent local review with findings: 16 stored findings
Review directory: /Users/mahesh/projects/mmani-demo-prs
Base branch: demo/agent-base

major [Functional Correctness]
→ tests/discounts.test.ts:46

Do not assert a negative order total.

This assertion makes a negative netCents the expected result for an
oversized fixed discount. Assert the chosen valid behavior instead: reject
the code or cap the discount so the net total is nonnegative. As per path
instructions, “Tests encode intended behavior.”


major [Functional Correctness]
→ tests/discounts.test.ts:26

Assert one deduction for the percent code.

The test’s 650-cent discount on a 6,500-cent subtotal should produce a
5,850-cent net total. The expected 5,200 cents accepts the
implementation’s second deduction through a negative order line. Correct
this assertion with the order-level discount implementation. As per path
instructions, “A test that asserts a wrong result … is a defect, not
evidence of correctness.”
```

CLI: F2 through the agent's tests (negative total; double deduction).

STEP 4 Inspect the same findings in VS Code

↑ Index

DO

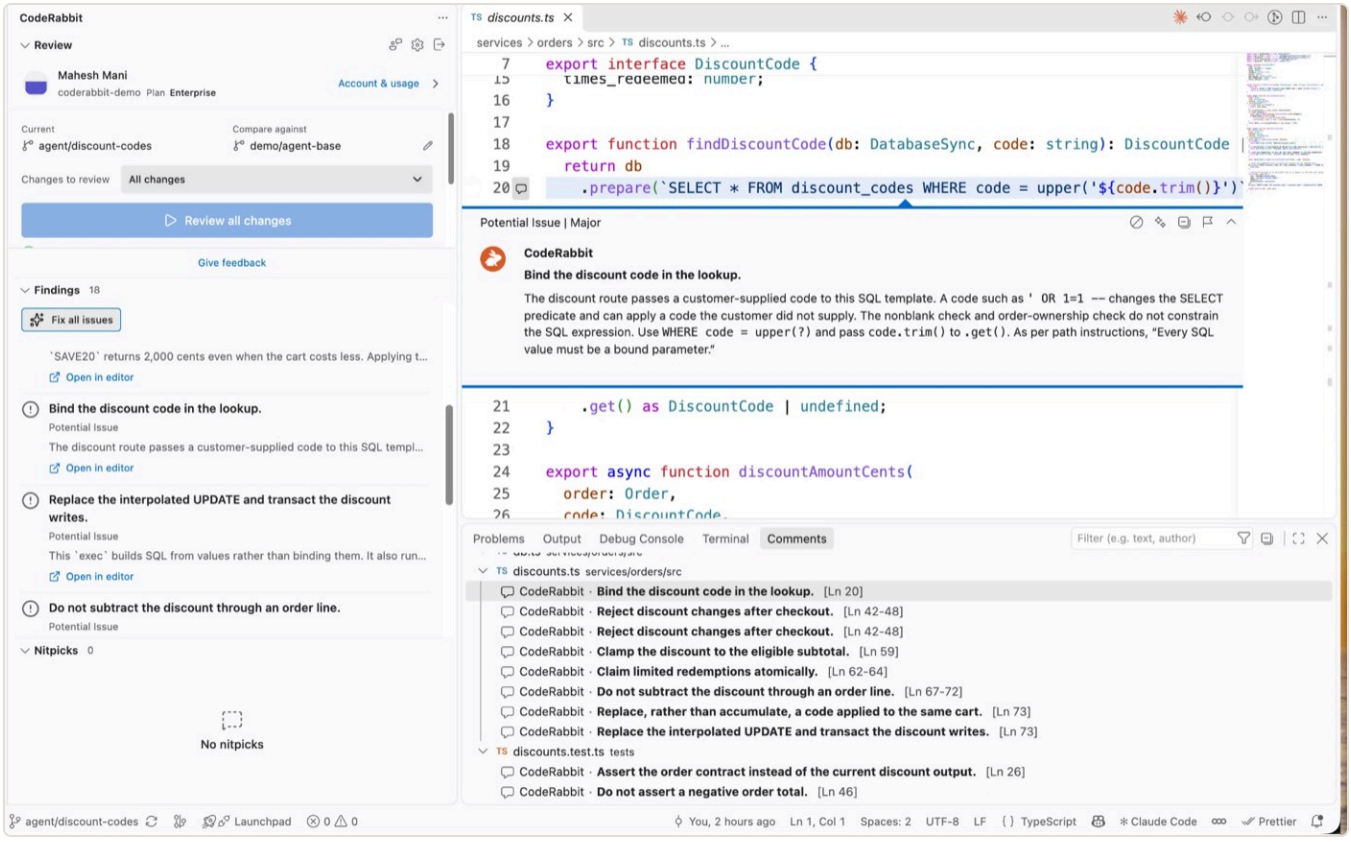
Run `code ~/projects/mmani-demo-prs`. In the CodeRabbit panel, set the base to `demo/agent-base` and the scope to **Review committed changes**. Open the SQL finding, then the discount-line finding.

SAY

"Same findings, in my editor, next to the code. I can apply the suggestion or hand it back to the agent with Fix with AI."

SEE

18 findings and **Fix all issues**. `F1` inline at line 20: "Bind the discount code in the lookup." `F2` in Comments: "Clamp the discount to the eligible subtotal" (Ln 59) and "Do not subtract the discount through an order line" (Ln 67-72). Leave fixes unapplied.



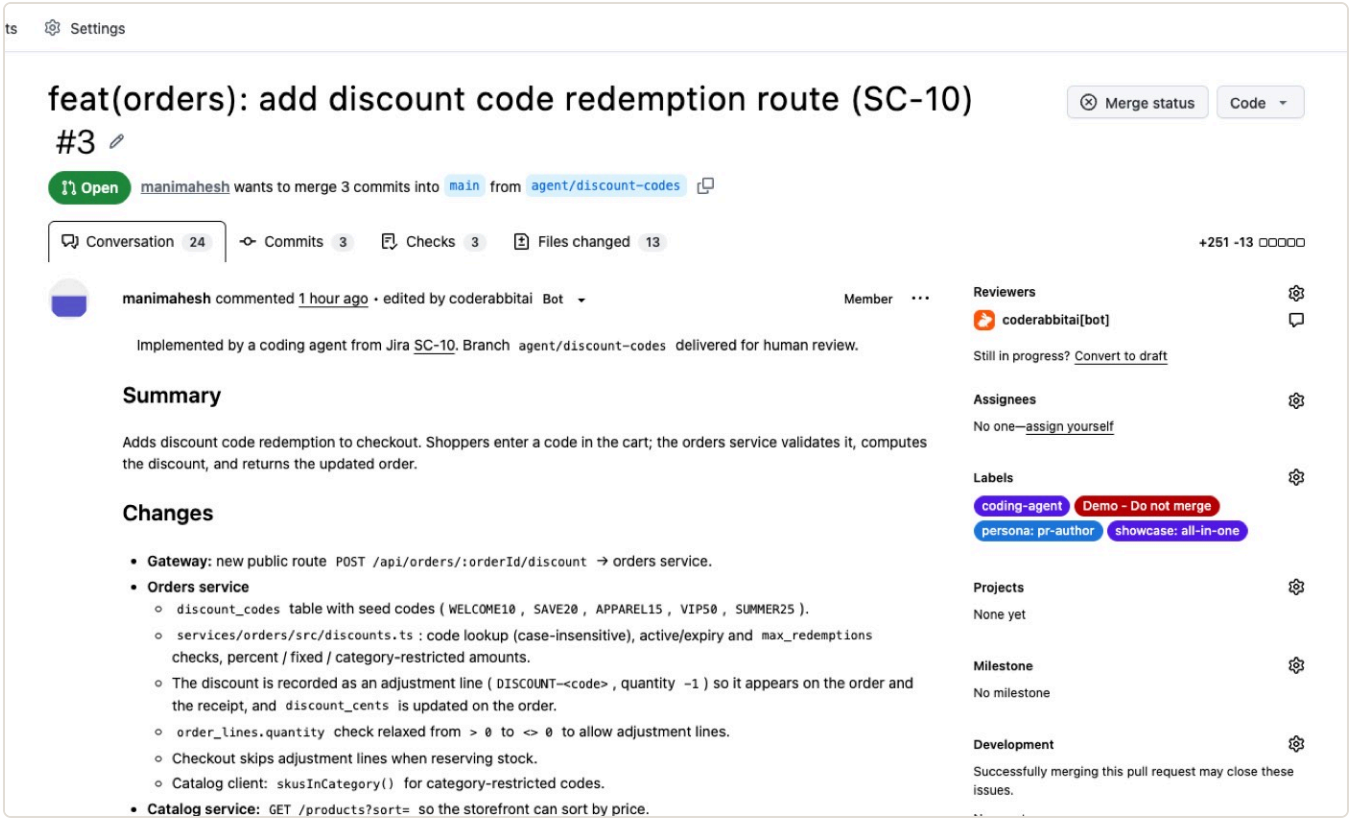
VS Code: F1 inline on the query, F2 in the findings and Comments lists.

> **DON'T DO THIS** Do not apply fixes locally during a recorded demo. The PR stops below depend on the defects staying in place.

Stage 2 · The pull request

STEP 5 **Open PR #3 and read the summary** ↑ Index

DO	SAY	SEE
Open PR #3. Point at the agent note, Jira link, and labels.	*"The developer pushed the agent's branch anyway. Now the whole team sees it — and CodeRabbit reviews it with more context than my laptop had: Jira, our org standards, and the repos that depend on this service."*	"Implemented by a coding agent from Jira SC-10", labels `coding-agent` and `Demo - Do not merge`.

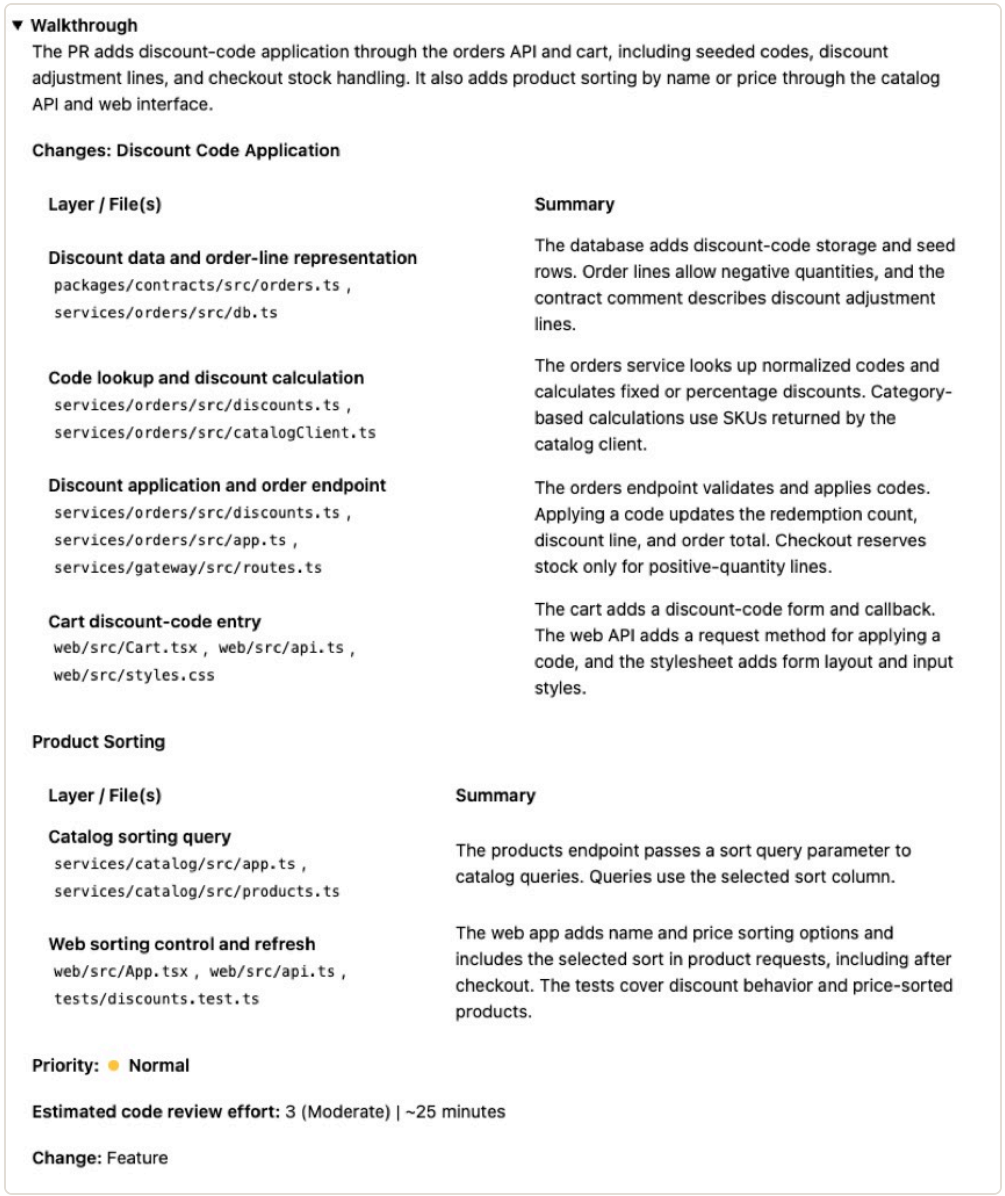


PR #3 header and description.

STEP 6 Walkthrough and sequence diagram

↑ Index

DO	SAY	SEE
In the CodeRabbit summary comment, expand **Walkthrough**. Scroll to the changes table and the sequence diagram.	*"Before any finding: here is what changed, grouped by purpose, and the request path from the cart through the gateway to the orders database. It also noticed product sorting, which nobody asked for."*	Grouped changes (Discount Code Application; Product Sorting), Priority, review effort 3 (~25 min), and the Shopper → Cart → Gateway → OrdersApp → OrdersDB diagram.



▼ **Walkthrough**

The PR adds discount-code application through the orders API and cart, including seeded codes, discount adjustment lines, and checkout stock handling. It also adds product sorting by name or price through the catalog API and web interface.

Changes: Discount Code Application

Layer / File(s)	Summary
Discount data and order-line representation `packages/contracts/src/orders.ts`, `services/orders/src/db.ts`	The database adds discount-code storage and seed rows. Order lines allow negative quantities, and the contract comment describes discount adjustment lines.
Code lookup and discount calculation `services/orders/src/discounts.ts`, `services/orders/src/catalogClient.ts`	The orders service looks up normalized codes and calculates fixed or percentage discounts. Category-based calculations use SKUs returned by the catalog client.
Discount application and order endpoint `services/orders/src/discounts.ts`, `services/orders/src/app.ts`, `services/gateway/src/routes.ts`	The orders endpoint validates and applies codes. Applying a code updates the redemption count, discount line, and order total. Checkout reserves stock only for positive-quantity lines.
Cart discount-code entry `web/src/Cart.tsx`, `web/src/api.ts`, `web/src/styles.css`	The cart adds a discount-code form and callback. The web API adds a request method for applying a code, and the stylesheet adds form layout and input styles.

Product Sorting

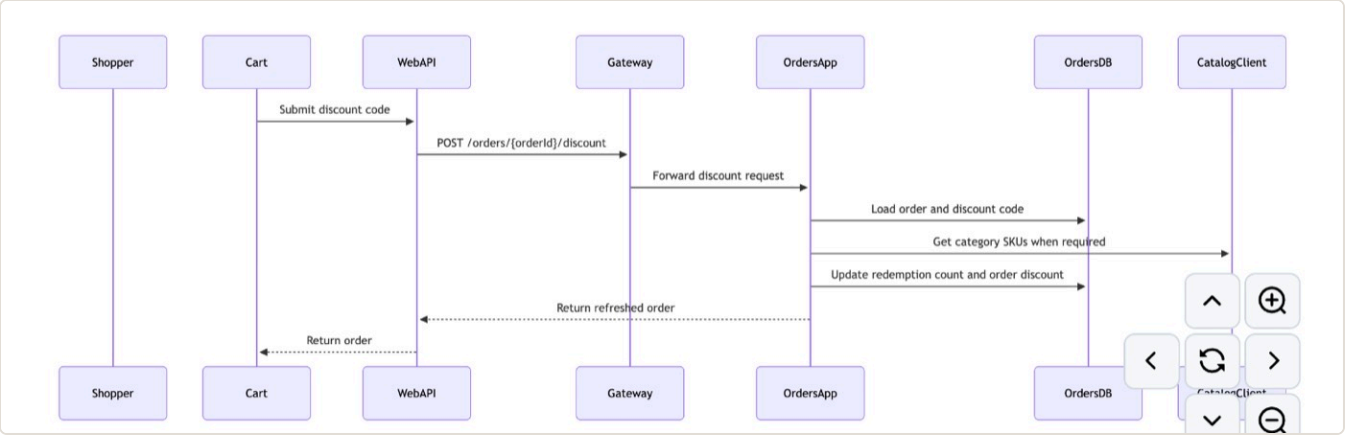
Layer / File(s)	Summary
Catalog sorting query `services/catalog/src/app.ts`, `services/catalog/src/products.ts`	The products endpoint passes a sort query parameter to catalog queries. Queries use the selected sort column.
Web sorting control and refresh `web/src/App.tsx`, `web/src/api.ts`, `tests/discounts.test.ts`	The web app adds name and price sorting options and includes the selected sort in product requests, including after checkout. The tests cover discount behavior and price-sorted products.

Priority: ● **Normal**

Estimated code review effort: 3 (Moderate) | ~25 minutes

Change: Feature

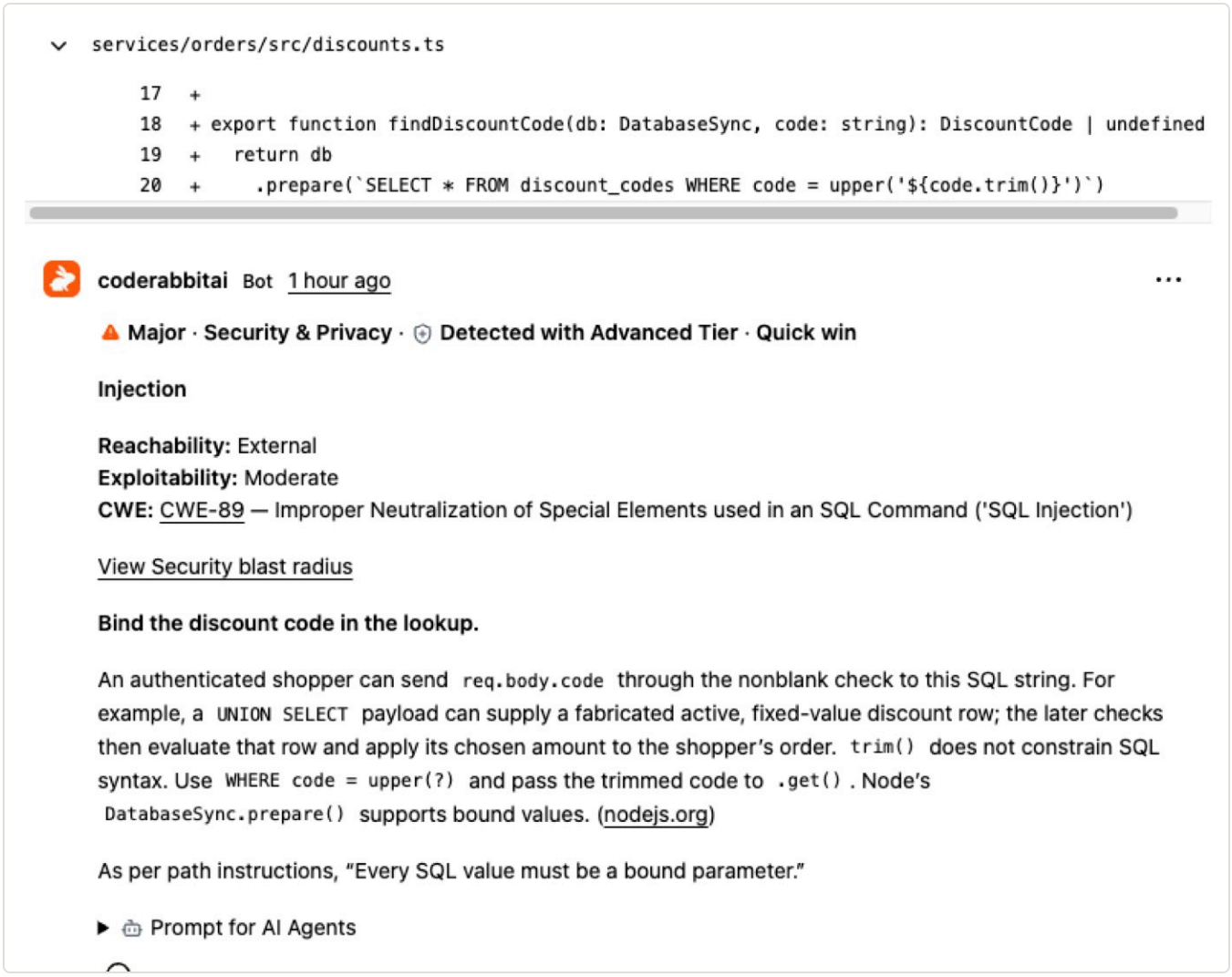
Walkthrough with layered changes.



Sequence diagram: the discount request path from the cart to the orders database.

STEP 7 F1 **The SQL injection finding** ↑ Index

DO	SAY	SEE
Open the discounts.ts:20 finding.	*"This is the same finding the CLI raised. Now it carries severity, category, reachability, exploitability, the CWE, a concrete exploit path, and the fix — bind the code as a parameter."*	Major · Security & Privacy · CWE-89 · Reachability External · Exploitability Moderate · *View Security blast radius*.



F1 on the PR: anatomy of a security finding.

STEP 8 F2 The money bug: counted twice, never capped

DO	SAY	SEE
Open the discounts.ts:73 finding, then the discounts.ts:30 finding.	*"The agent stored the discount twice — as a minus-one line and on the order — so a $65 cart with 10% off charges $52, not $58.50. And a $20 code on a $15 mug produces a negative total."*	"Subtract the discount only once" with the exact cents; "Cap a fixed discount at the eligible subtotal", Sources: Path instructions, Linked repositories.



F2: the discount is subtracted twice.

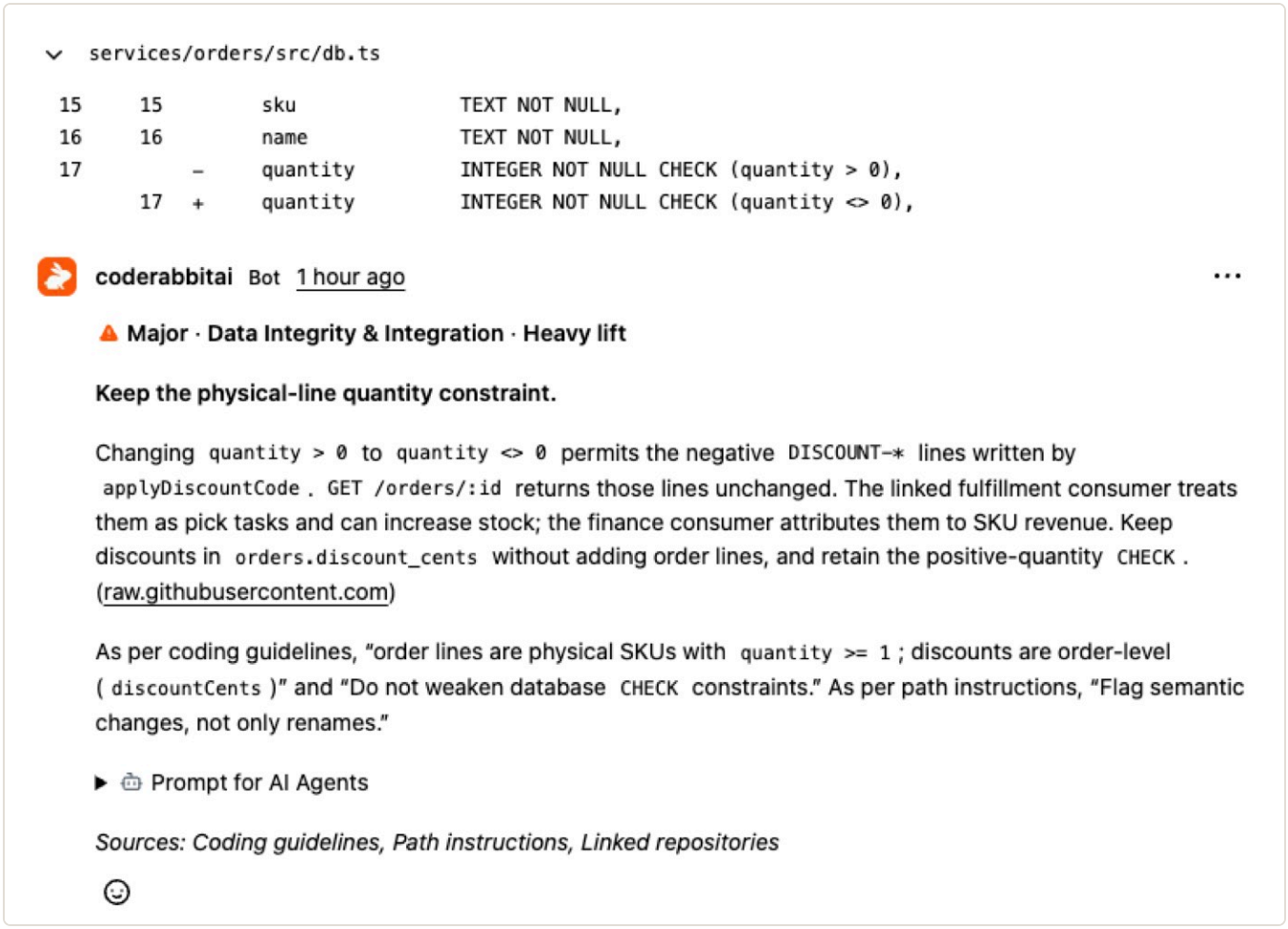


F2: uncapped fixed discount gives a negative net.

STEP 9 F2 It breaks another repository

↑ Index

DO	SAY	SEE
Open the db.ts:17 finding.	*"To make its design fit, the agent relaxed a database constraint. CodeRabbit read the fulfillment repo: those minus-one lines become pick tasks and increase stock, and finance books them as SKU revenue."*	"Keep the physical-line quantity constraint." Sources: **Coding guidelines, Path instructions, Linked repositories**.



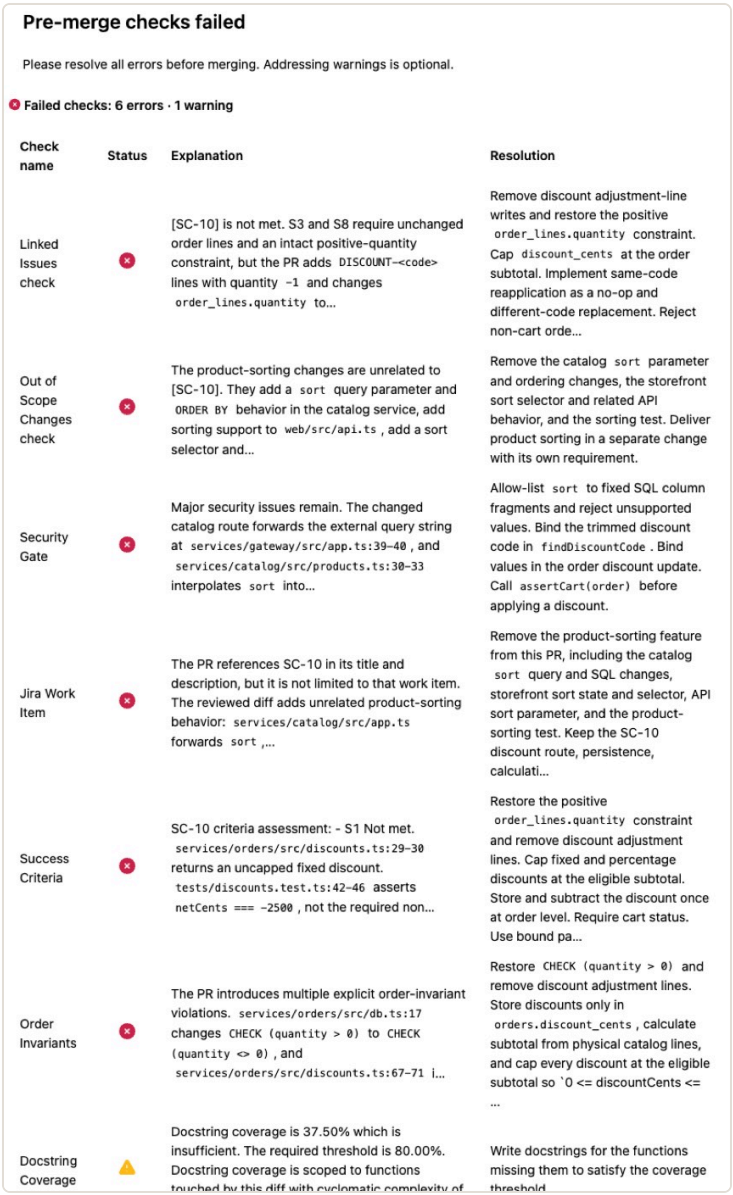
F2 reaches the linked consumer repository.

WOW Pause on the Sources line. One finding cites the org's coding standards (a separate repo), the path instructions in this repo's config, and the downstream repo it would break.

STEP 10 Pre-merge checks against Jira

↑ Index

DO	SAY	SEE
Scroll to **Pre-merge checks** in the summary comment. Expand *Success Criteria*.	*"CodeRabbit read SC-10 and graded the PR against every success criterion — with file and line evidence. It also caught the out-of-scope sorting feature."*	6 errors · 1 warning: Linked Issues, Out of Scope, Security Gate, Jira Work Item, Success Criteria (S1–S10 Not met), Order Invariants; Docstring Coverage warning.



Pre-merge checks failed

Please resolve all errors before merging. Addressing warnings is optional.

Failed checks: 6 errors · 1 warning

Check name	Status	Explanation	Resolution
Linked Issues check	✗	[SC-10] is not met. S3 and S8 require unchanged order lines and an intact positive-quantity constraint, but the PR adds `DISCOUNT-<code>` lines with quantity `-1` and changes `order_lines.quantity` to...	Remove discount adjustment-line writes and restore the positive `order_lines.quantity` constraint. Cap `discount_cents` at the order subtotal. Implement same-code reapplication as a no-op and different-code replacement. Reject non-cart orde...
Out of Scope Changes check	✗	The product-sorting changes are unrelated to [SC-10]. They add a `sort` query parameter and `ORDER BY` behavior in the catalog service, add sorting support to `web/src/api.ts`, add a sort selector and...	Remove the catalog `sort` parameter and ordering changes, the storefront sort selector and related API behavior, and the sorting test. Deliver product sorting in a separate change with its own requirement.
Security Gate	✗	Major security issues remain. The changed catalog route forwards the external query string at `services/gateway/src/app.ts:39-40`, and `services/catalog/src/products.ts:30-33` interpolates `sort` into...	Allow-list `sort` to fixed SQL column fragments and reject unsupported values. Bind the trimmed discount code in `findDiscountCode`. Bind values in the order discount update. Call `assertCart(order)` before applying a discount.
Jira Work Item	✗	The PR references SC-10 in its title and description, but it is not limited to that work item. The reviewed diff adds unrelated product-sorting behavior: `services/catalog/src/app.ts` forwards `sort`,...	Remove the product-sorting feature from this PR, including the catalog `sort` query and SQL changes, storefront sort state and selector, API sort parameter, and the product-sorting test. Keep the SC-10 discount route, persistence, calculati...
Success Criteria	✗	SC-10 criteria assessment: - S1 Not met. `services/orders/src/discounts.ts:29-30` returns an uncapped fixed discount. `tests/discounts.test.ts:42-46` asserts `netCents === -2500`, not the required non...	Restore the positive `order_lines.quantity` constraint and remove discount adjustment lines. Cap fixed and percentage discounts at the eligible subtotal. Store and subtract the discount once at order level. Require cart status. Use bound pa...
Order Invariants	✗	The PR introduces multiple explicit order-invariant violations. `services/orders/src/db.ts:17` changes `CHECK (quantity > 0)` to `CHECK (quantity <> 0)`, and `services/orders/src/discounts.ts:67-71` i...	Restore `CHECK (quantity > 0)` and remove discount adjustment lines. Store discounts only in `orders.discount_cents`, calculate subtotal from physical catalog lines, and cap every discount at the eligible subtotal so `0 <= discountCents <= ...
Docstring Coverage	⚠	Docstring coverage is 37.50% which is insufficient. The required threshold is 80.00%. Docstring coverage is scoped to functions touched by this diff with cyclomatic complexity of	Write docstrings for the functions missing them to satisfy the coverage threshold

Pre-merge checks, including the Jira success-criteria gate.

> **DON'T DO THIS** Do not describe green CI as proof. CI passes because the agent's tests assert the wrong totals. The checks and the findings are different signals.

Stage 3 · The reviewer: Triage and Change Stack

STEP 11 Triage: where does PR #3 sit?

↑ Index

DO

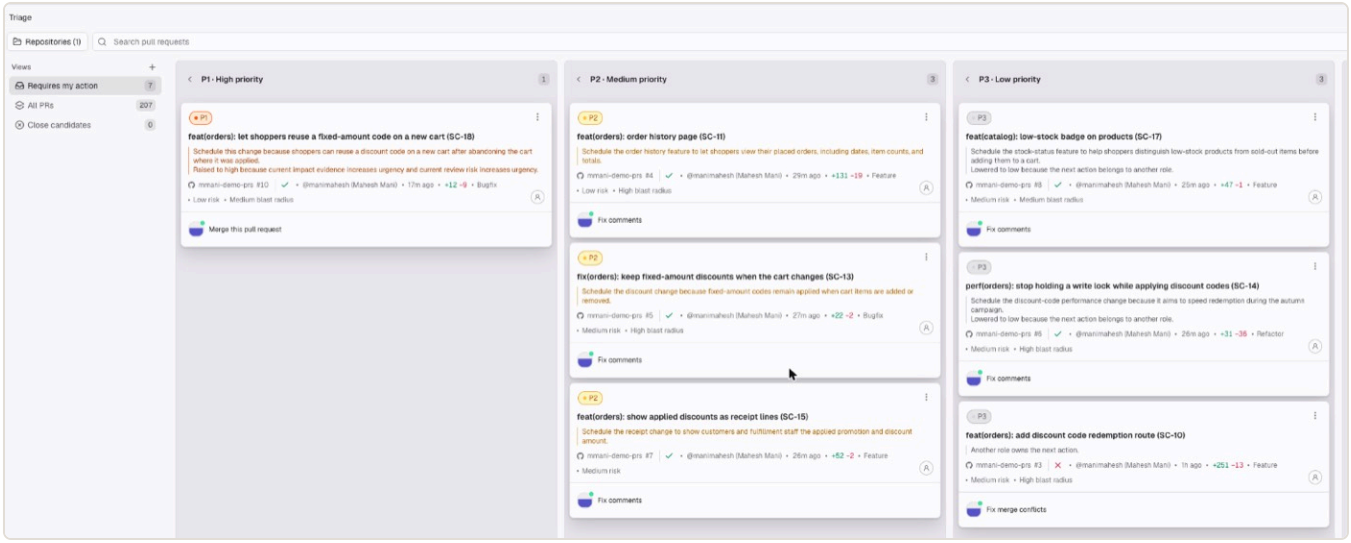
Open Triage filtered to `mmani-demo-prs`. Find PR #3 in the P3 column.

SAY

"As a reviewer I see the whole queue, prioritized, with risk and blast radius on every card. PR #3 tells me another role owns the next action — the author has merge conflicts to fix."

SEE

P1/P2/P3 columns; PR #3 card with Medium risk, High blast radius, and "Fix merge conflicts".

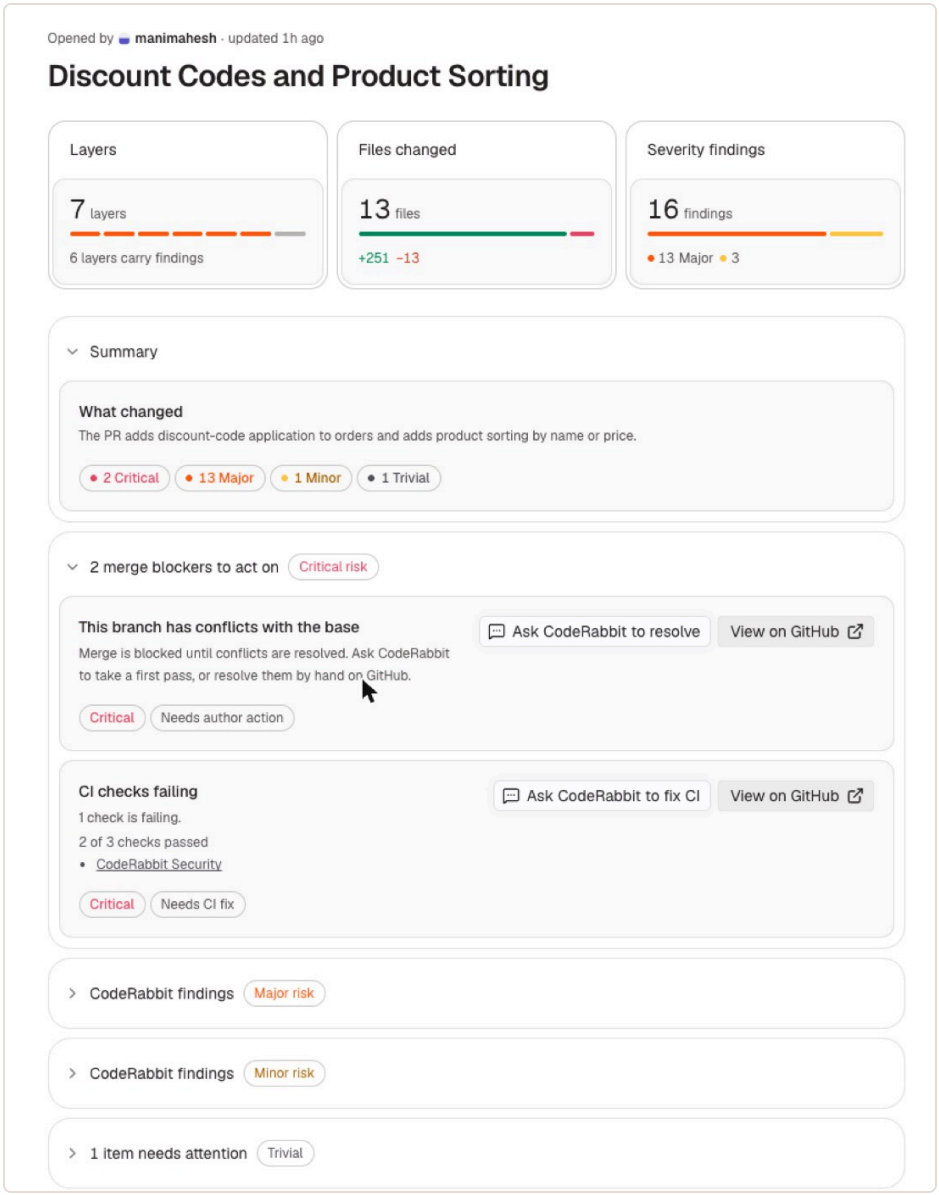


Triage queue for mmani-demo-prs.

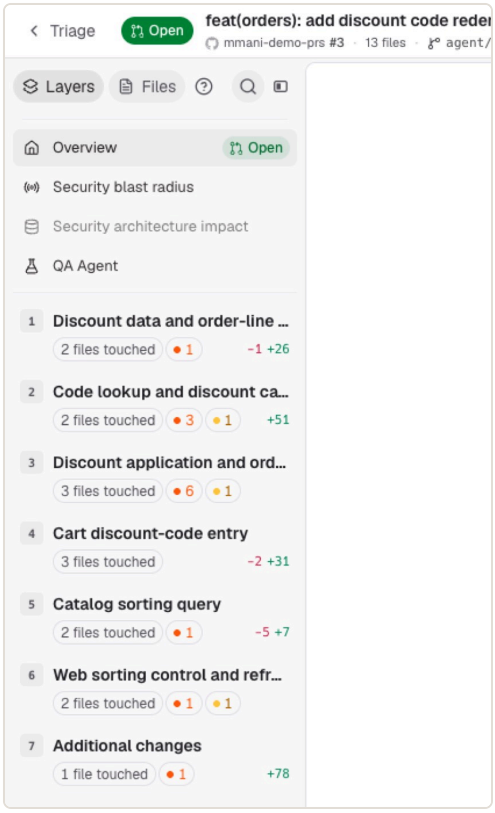
STEP 12 Change Stack overview and layers

↑ Index

DO	SAY	SEE
Open Change Stack for PR #3. Read the cards, then the layer rail.	*"Seven functional layers instead of thirteen files. Six of them carry findings, and the overview tells me what blocks the merge."*	7 layers, 13 files, 16 findings (2 Critical, 13 Major); merge blockers; the layer rail with per-layer finding counts.



Change Stack overview.

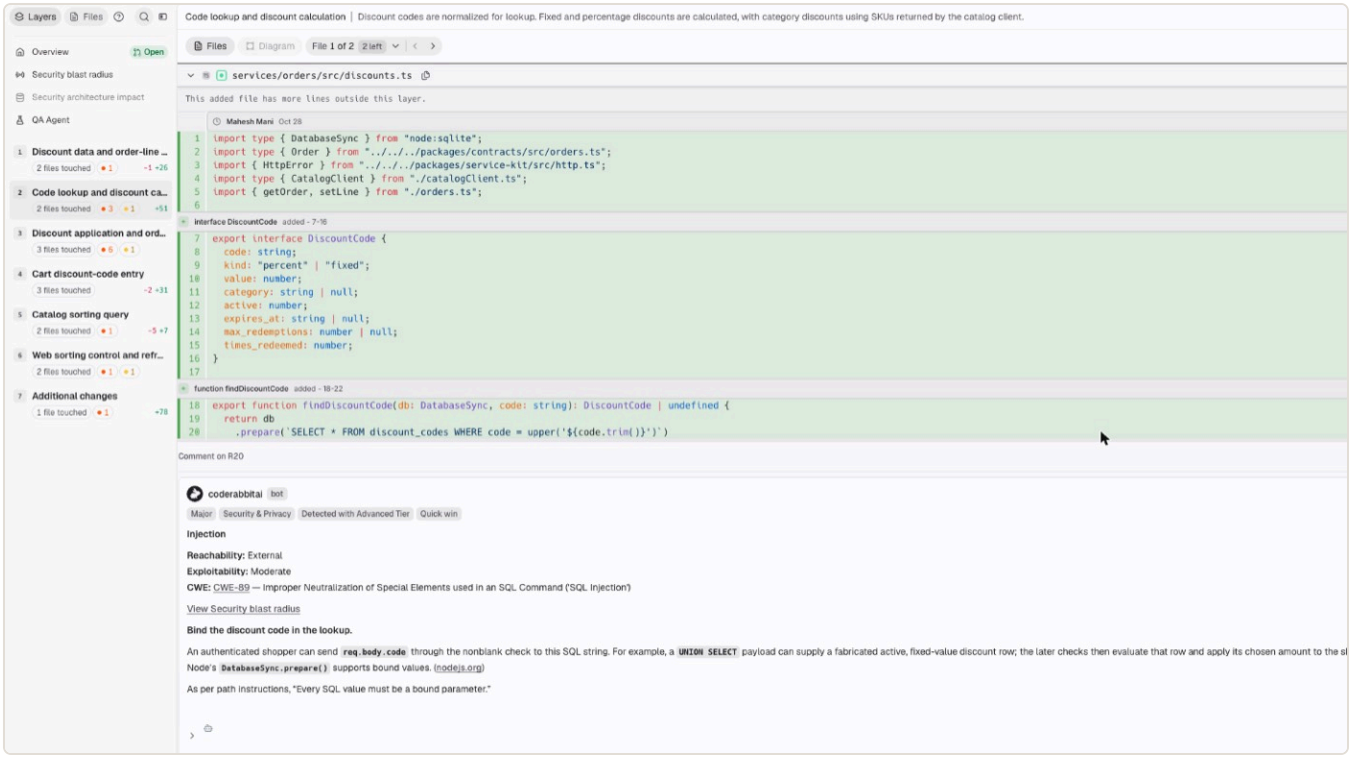

Triage
Open
feat(orders): add discount code reder
mmani-demo-prs #3
13 files
agent/
Layers
Files
Overview
Open
Security blast radius
Security architecture impact
QA Agent
1
Discount data and order-line ...
2 files touched
1
-1 +26
2
Code lookup and discount ca...
2 files touched
3
1
+51
3
Discount application and ord...
3 files touched
6
1
4
Cart discount-code entry
3 files touched
-2 +31
5
Catalog sorting query
2 files touched
1
-5 +7
6
Web sorting control and refr...
2 files touched
1
1
7
Additional changes
1 file touched
1
+78

Layer rail.

STEP 13 F1 The finding inside its layer

↑ Index

DO	SAY	SEE
Select layer 2, *Code lookup and discount calculation*.	*"Same SQL injection, now in context: the lookup function, the interface it returns, and the finding pinned to line 20."*	The layer summary, the diff of `discounts.ts`, and F1 inline with its CWE and fix.

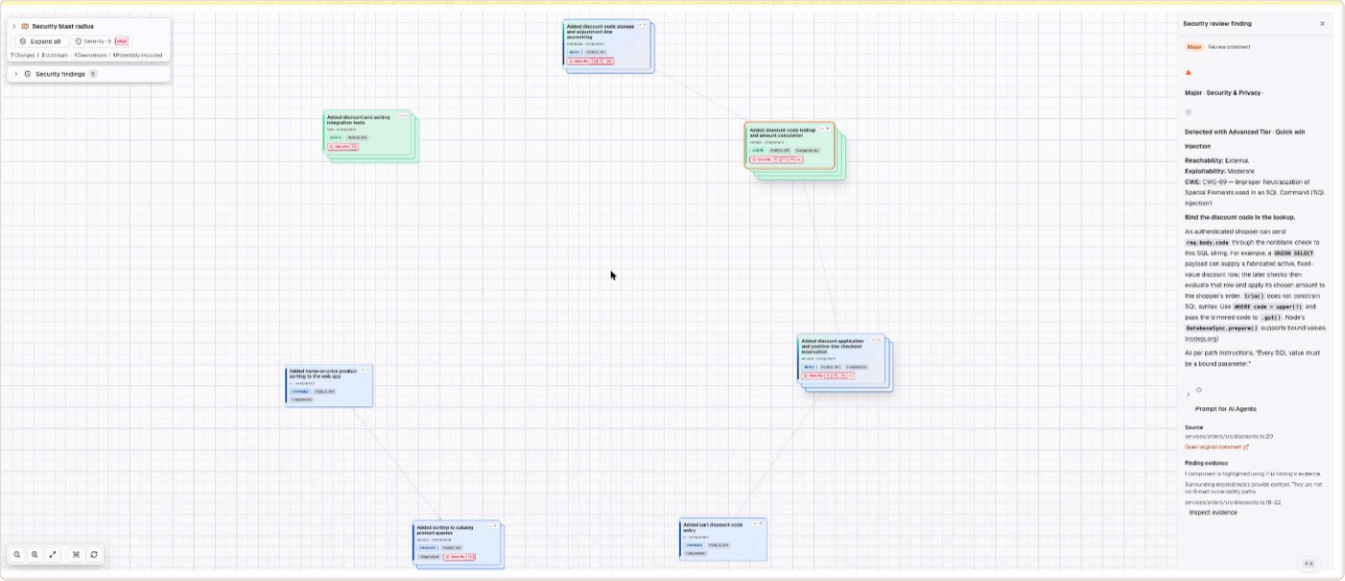


F1 in Change Stack layer 2.

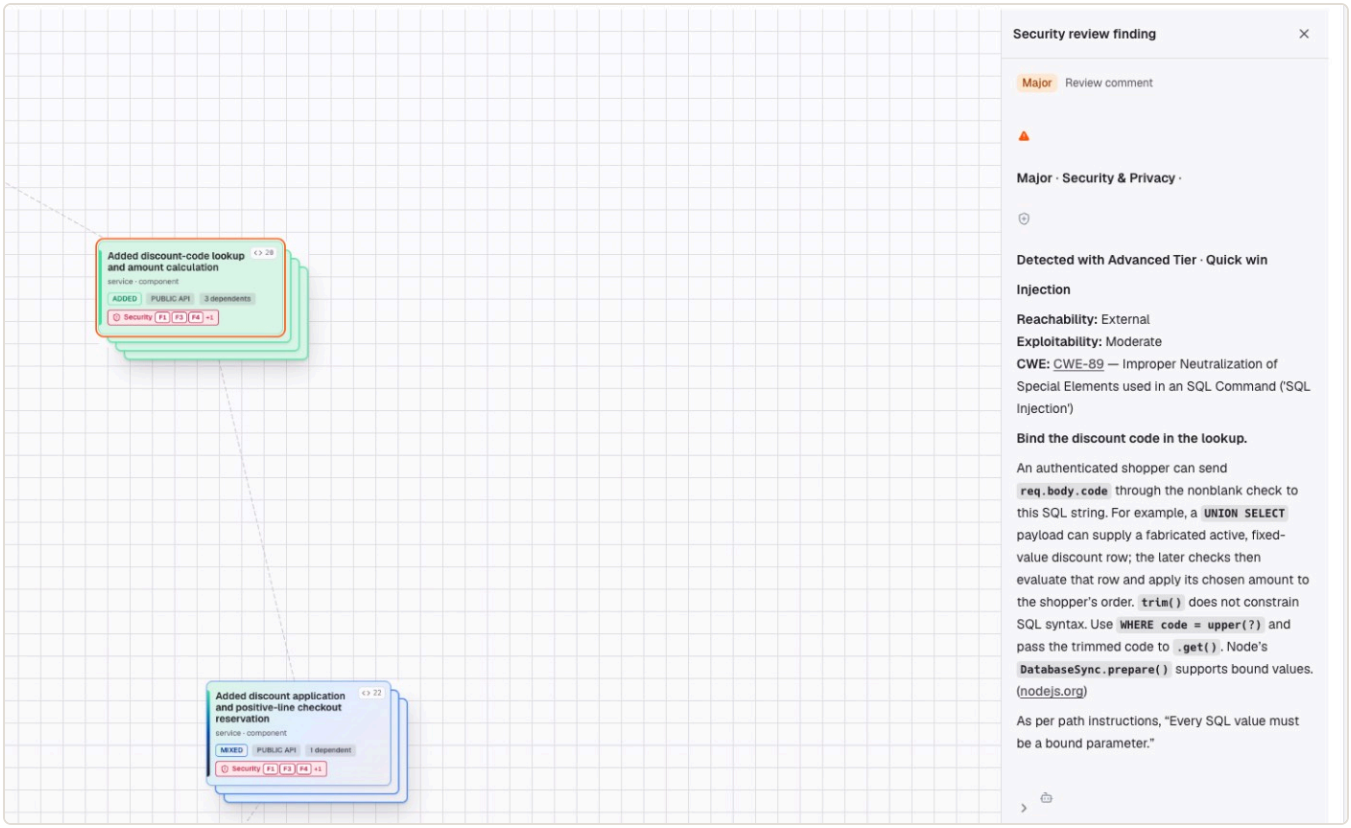
STEP 14 F1 Security Blast Radius

↑ Index

DO	SAY	SEE
From the F1 comment, click **View Security blast radius**. Select the highlighted lookup component.	*"The vulnerable line is the start. Which components does it reach? The lookup feeds discount application and checkout reservation."*	7 changed, 3 upstream, 1 downstream, 1 potentially impacted; the F1 panel on the right with evidence.



Security Blast Radius graph for F1.

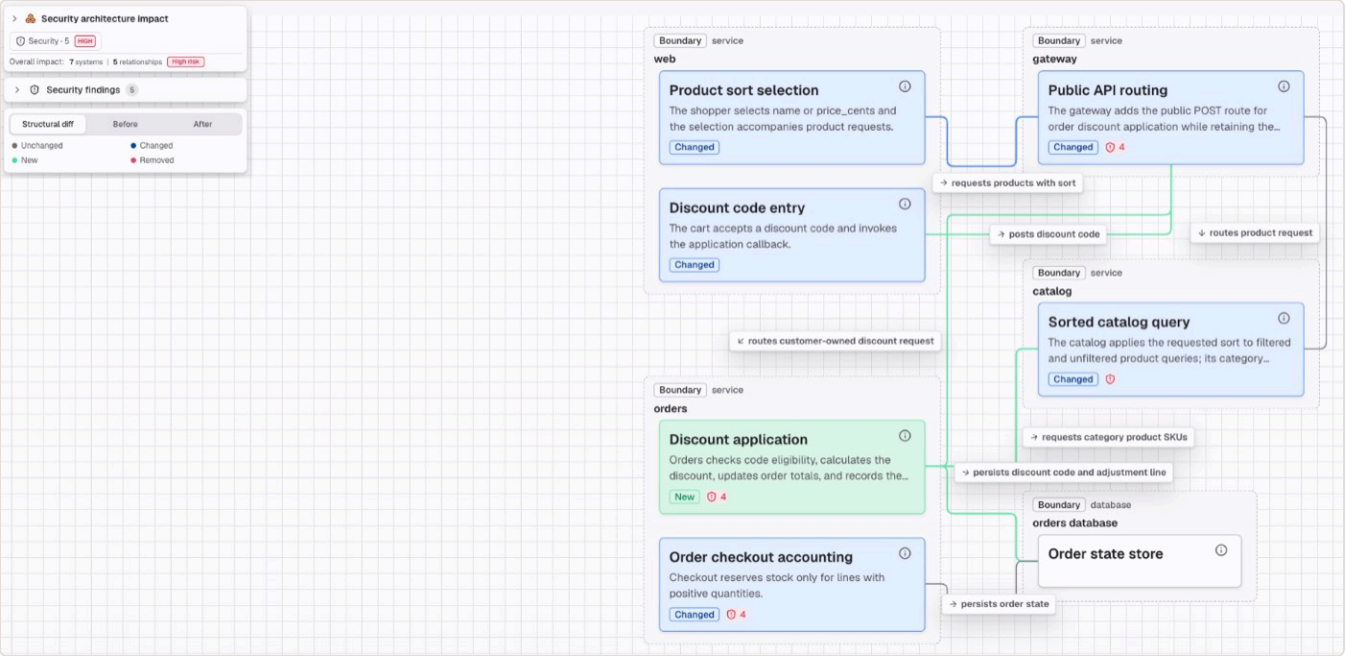


F1 selected: highlighted component and finding detail.

STEP 15 F1 F2 Security architecture impact

↑ Index

DO	SAY	SEE
Click **Security architecture impact** in the rail. Then, on GitHub, expand *Security architecture risk* in the summary.	*"Architecture view: customer input now crosses from the gateway into SQL in two services, and discount application is a new component writing order state. Both of our findings are listed as retained high concerns."*	Boundaries for web, gateway, catalog, orders, and the orders database; overall impact High risk. Retained concerns: SQL from shopper code (F1); negative and double-counted totals (F2).



Security architecture impact in Change Stack.

▼ **Security architecture risk:** ⚠ **High** · up to `f8c4f`

Customer input newly reaches SQL construction in both discount redemption and product sorting. Discounts can also change placed orders, reduce totals twice, and consume shared redemption allowances without an atomic update. Ownership checks limit ordinary order access, but they do not preserve these financial and lifecycle guarantees.

Retained concerns

- **High · security · observed:** The new discount lookup inserts shopper-controlled code directly into SQL. An owned-order check does not constrain which discount row the query selects, so code selection is no longer an effective boundary for promotion eligibility or value.
- **High · security · observed:** The gateway preserves the caller's sort query, and catalog inserts it into ORDER BY without an allowlist. Authentication does not prevent this new caller-to-SQL syntax transition. Both filtered and unfiltered product-list queries are affected; database writes or broader service compromise are not established.
- **High · security · observed:** The discount endpoint enforces ownership but omits the cart-state guard used by existing line edits and checkout. Customers can therefore change adjustment lines, totals, and redemption counts for their already placed orders, breaking the terminal-state boundary relied on by downstream readers.
- **High · security · observed:** The new writer represents each discount both as a negative line and as an accumulated order-level amount, while pricing subtracts both. Repetition overwrites the same adjustment line but keeps increasing discount_cents, and fixed discounts are uncapped. Customers can produce understated or negative authoritative totals that violate the documented fulfillment and finance contract; actual payment or refund effects are not established.
- **Medium · security · inferred:** Redemption checks, the shared counter increment, adjustment insertion, and order-total update are separate operations without an atomic or repeat-safe redemption record. Requests can pass the same limit before either increments it, and failure after the increment can consume a shared allowance without completing the discount. Existing persisted schemas retain the old positive-quantity constraint, providing a concrete failure path during upgrade.

▶ **Security review details**

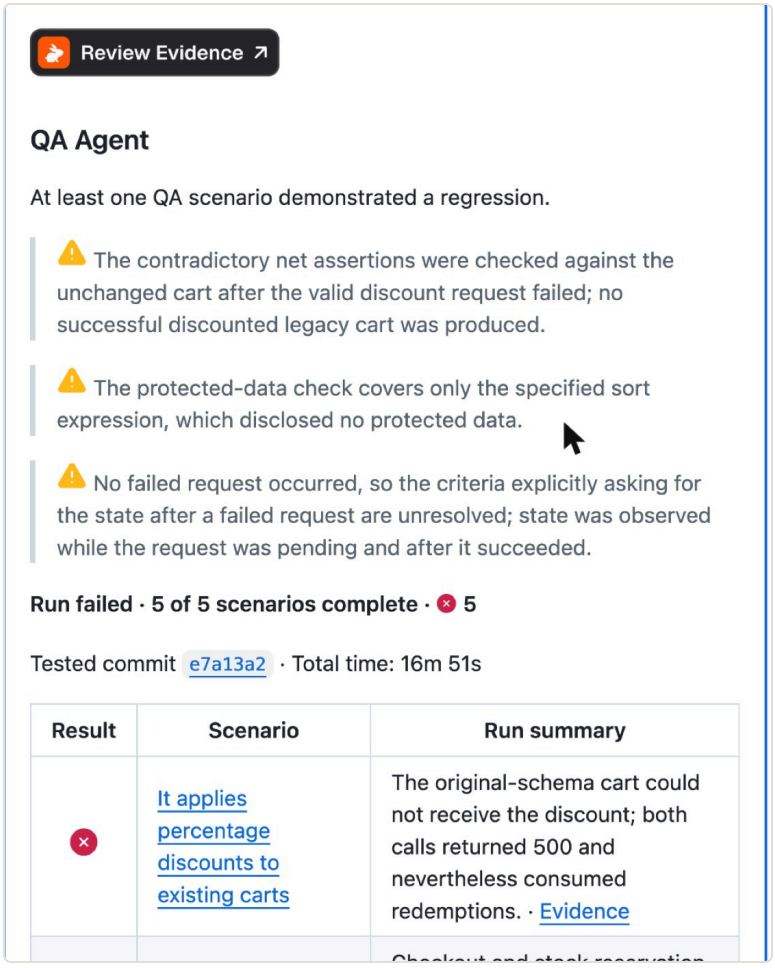
Security architecture risk in the PR summary.

Stage 4 · QA Agent

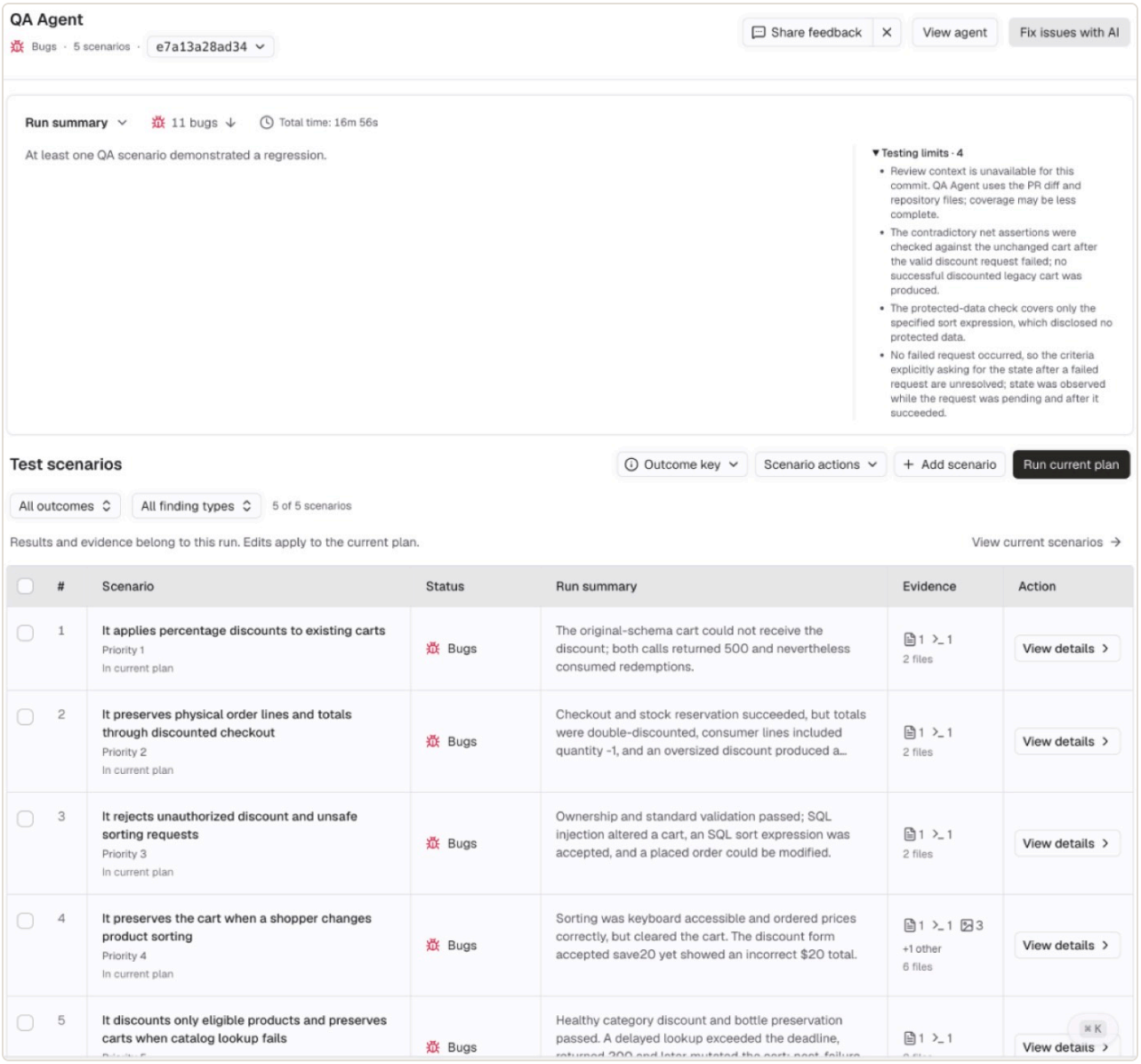
The QA Agent ran against the deployed demo environment on commit `e7a13a2`: 5 scenarios, 11 bugs, 16 m 56 s. All five scenarios failed. The two below carry F1 and F2.

STEP 16 **Run the QA Agent on PR #3** ↑ Index

DO	SAY	SEE
On PR #3, comment `@coderabbitai test` (or open **QA Agent** in Change Stack and click **Run current plan**). For the prepared demo, open the existing run from the PR comment's **Review Evidence** button.	*"Findings are claims. The QA Agent turns them into tests, runs them against the running app, and keeps the evidence."*	"At least one QA scenario demonstrated a regression." Run failed · 5 of 5 scenarios · 11 bugs, with testing limits stated plainly.



QA Agent result on the PR.

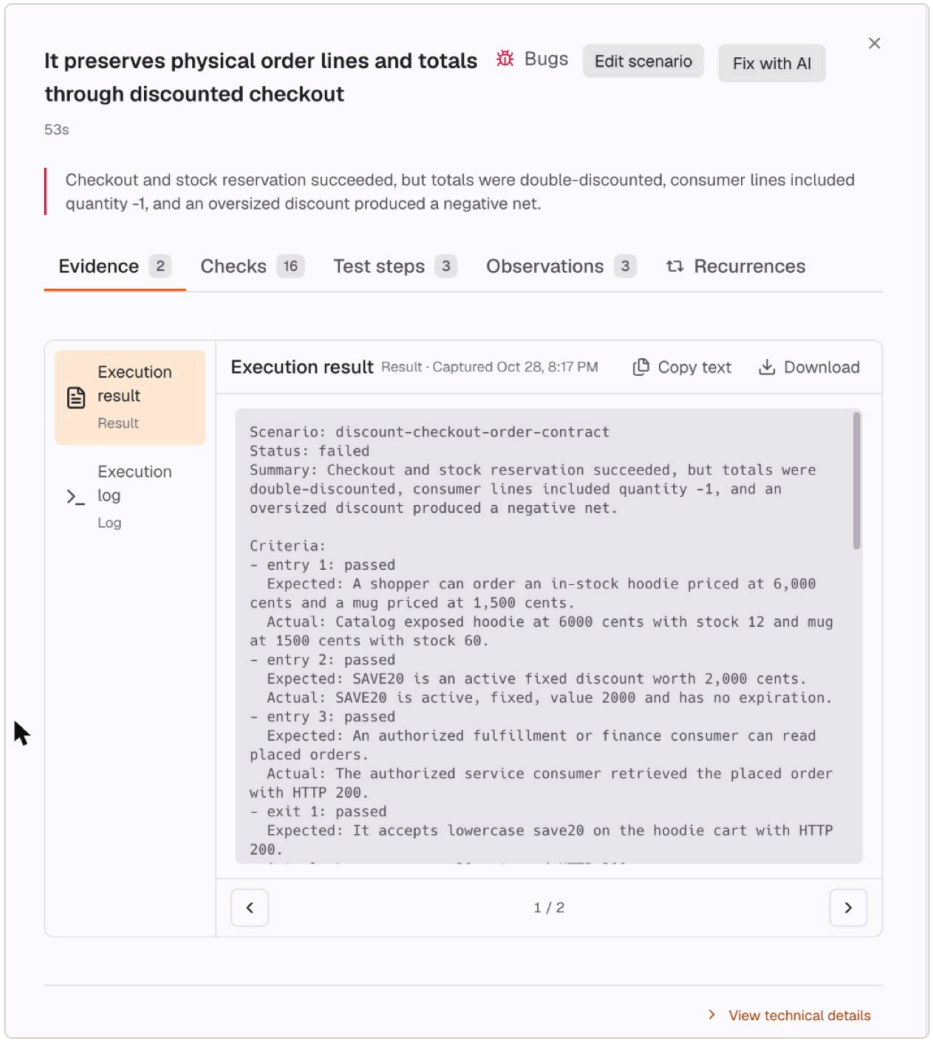


The run in Change Stack: five scenarios, each with evidence.

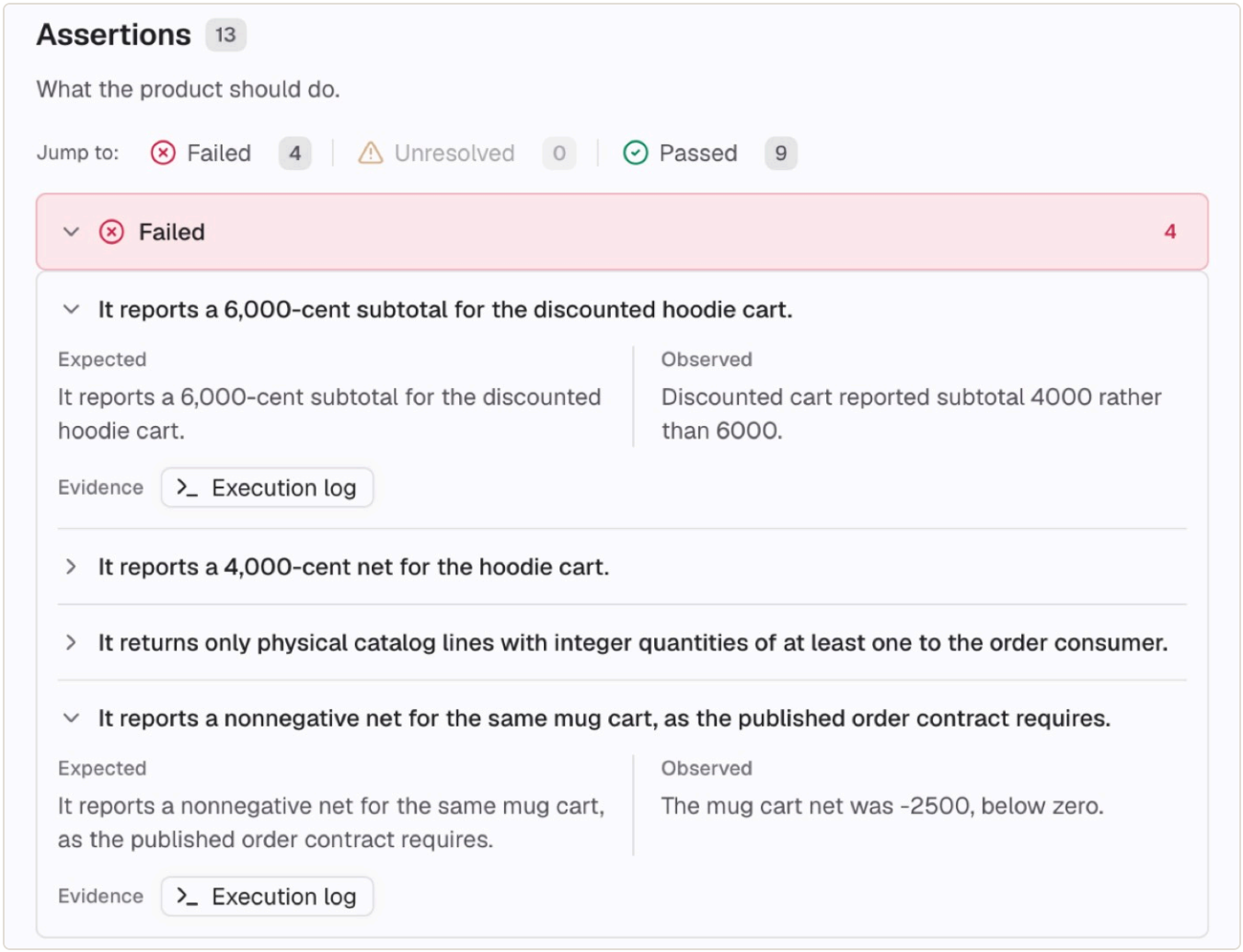
STEP 17 F2 The money bug, proven in the running app

↑ Index

DO	SAY	SEE
Open scenario 2, *It preserves physical order lines and totals through discounted checkout*. Show **Evidence**, then **Checks** → Failed.	*"Checkout worked — and that is the problem. The hoodie cart reports a 4,000-cent subtotal instead of 6,000, the fulfillment consumer receives a quantity of minus one, and the mug cart nets minus $25."*	Observed: "Discounted cart reported subtotal 4000 rather than 6000." "The mug cart net was -2500, below zero." Execution log linked to each check.



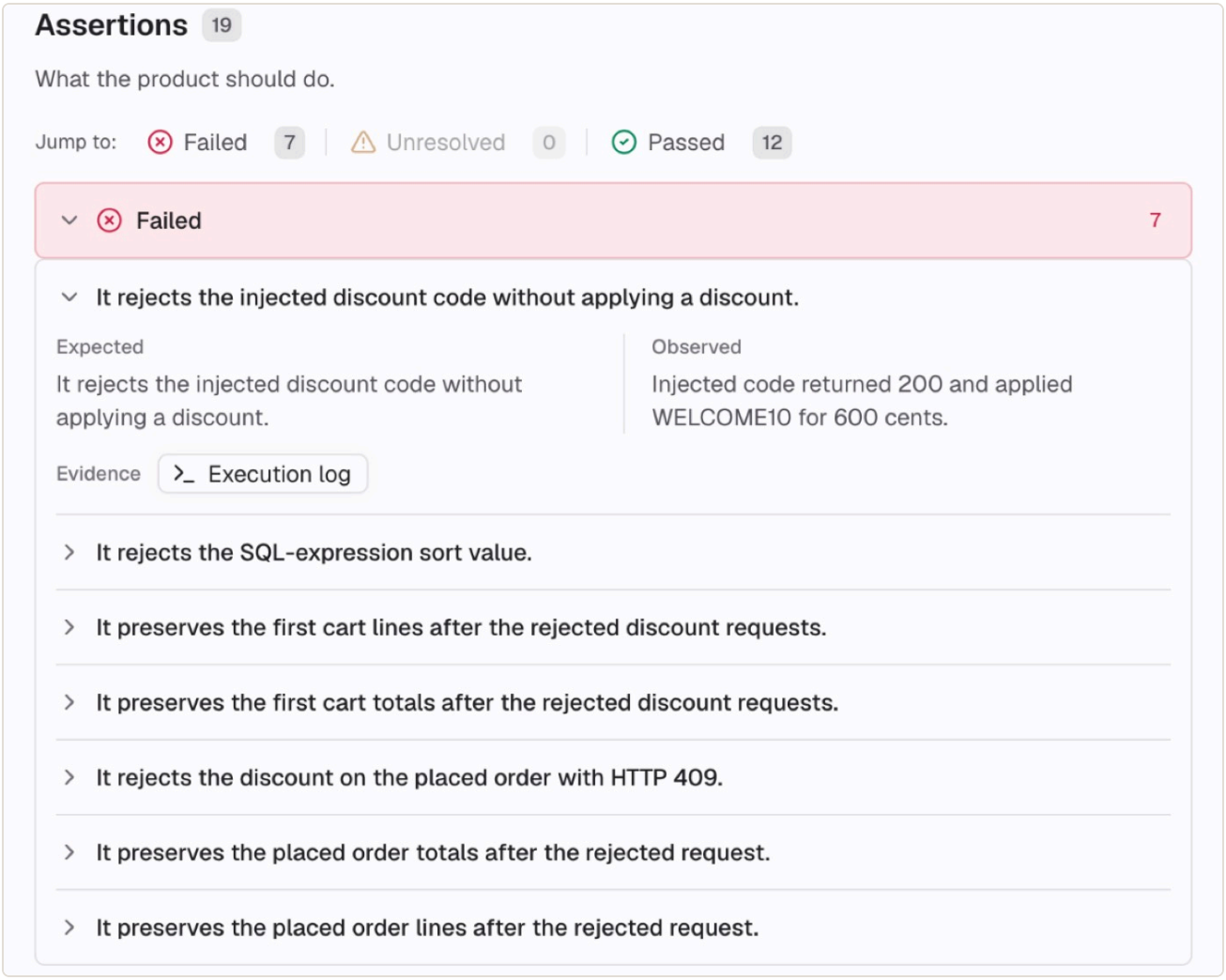
Scenario 2 evidence: execution result.



Scenario 2 failed checks: double-counted subtotal and negative net.

STEP 18 F1 The injection, exploited in the running app

DO	SAY	SEE
Open scenario 3, *It rejects unauthorized discount and unsafe sorting requests*. Show **Checks** → Failed.	*"This is F1 with proof. An injected code was accepted and applied someone else's discount. Same scenario: the SQL sort expression was accepted and a placed order was changed."*	Observed: "Injected code returned 200 and applied WELCOME10 for 600 cents." 7 failed assertions, each with an execution log.



Scenario 3 failed checks: the injected code was accepted.

WOW The same two findings from the CLI are now reproduced against the running application, with expected versus observed values and execution logs.

Stage 5 · Application Security: Deep Scan

Deep Scan `2b10cbc0` of `agent/discount-codes`, October 28, 2026: 10 findings (1 high, 5 medium, 4 low). Scan the agent branch, not `main`: `main` holds the corrected code and does not contain F1.

STEP 19 F1 Deep Scan ranks the same injection highest

↑ Index

DO

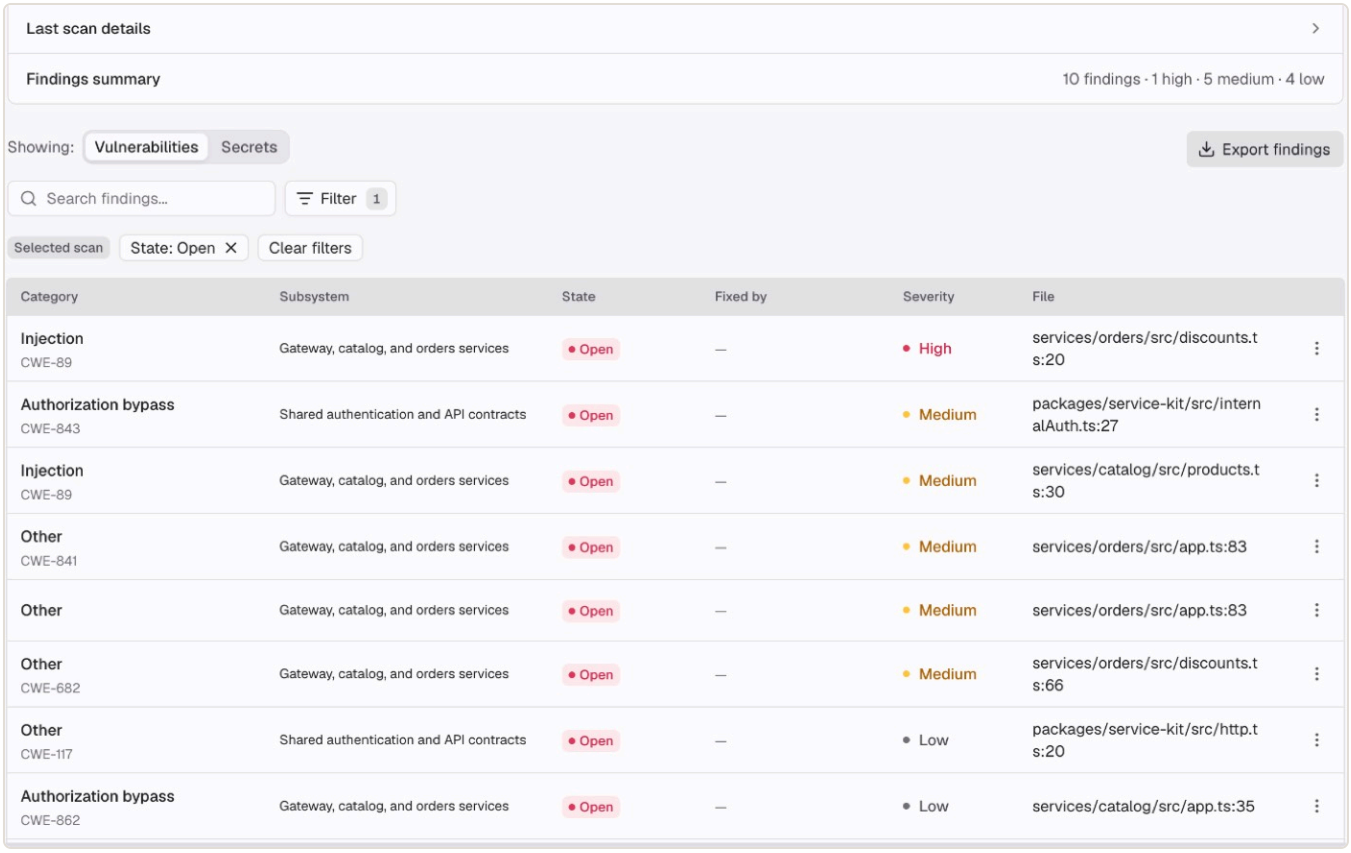
Open CodeRabbit Security → Repositories → `mmani-demo-prs` → Findings, with the agent-branch scan selected. Do not start a new scan during the demo.

SAY

"The security team doesn't read PRs. They scan the repository — and the one High finding is the same line we started with on my laptop."

SEE

Injection · CWE-89 · **High** · `services/orders/src/discounts.ts:20`. Also F2 as CWE-682 at `discounts.ts:66`, the negative discount line.



Deep Scan findings for the agent branch.

STEP 20 F1 Reachability from the public route to the sink

↑ Index

DO

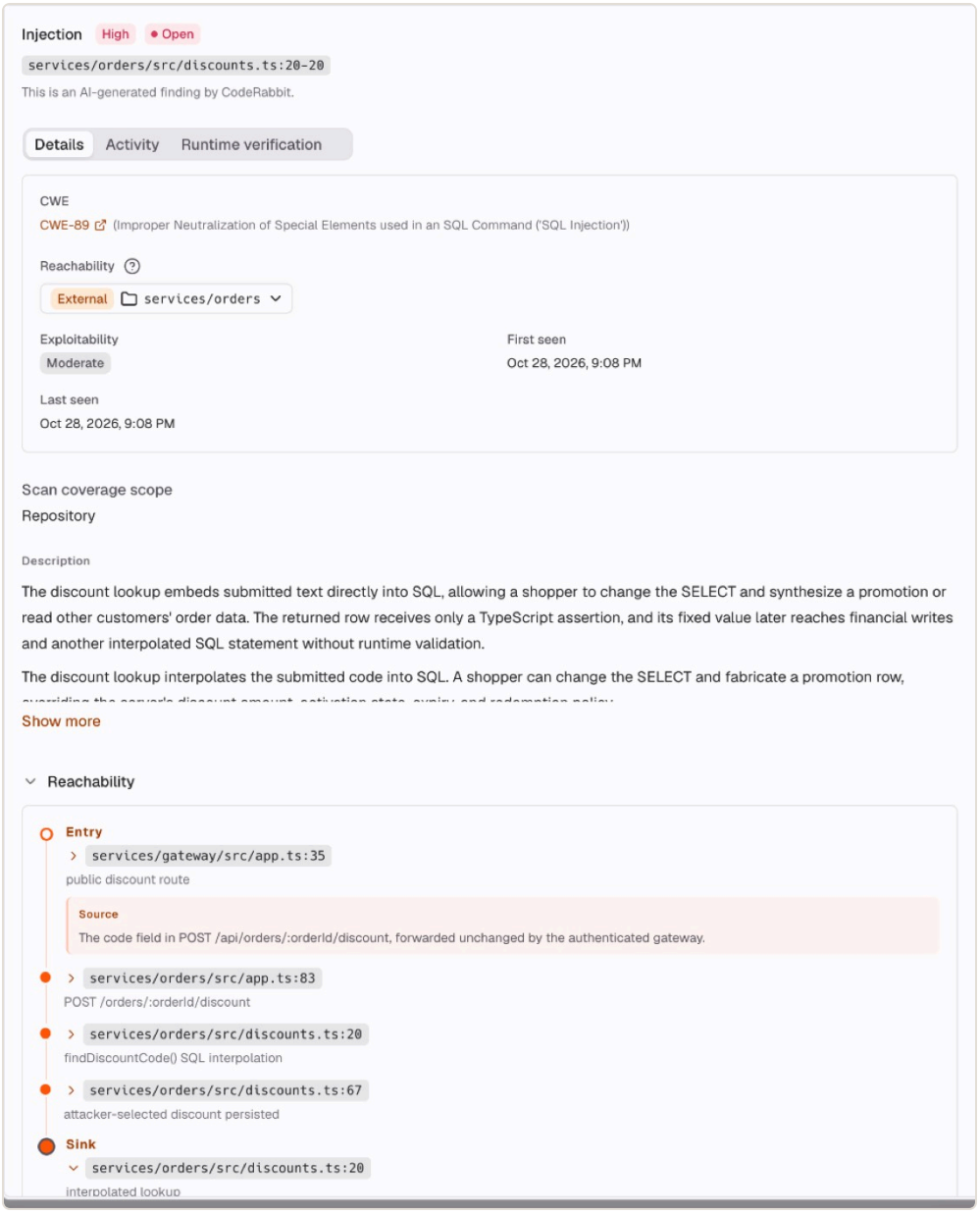
Select the High injection finding. Scroll to **Reachability**.

SAY

"It traces the attack path: the public discount route on the gateway, the orders handler, the interpolated lookup, and where the attacker-selected discount gets written."

SEE

Reachability External · Exploitability Moderate. Entry `gateway/src/app.ts:35` → `orders/src/app.ts:83` → `discounts.ts:20` → `discounts.ts:67`; sink `discounts.ts:20`.



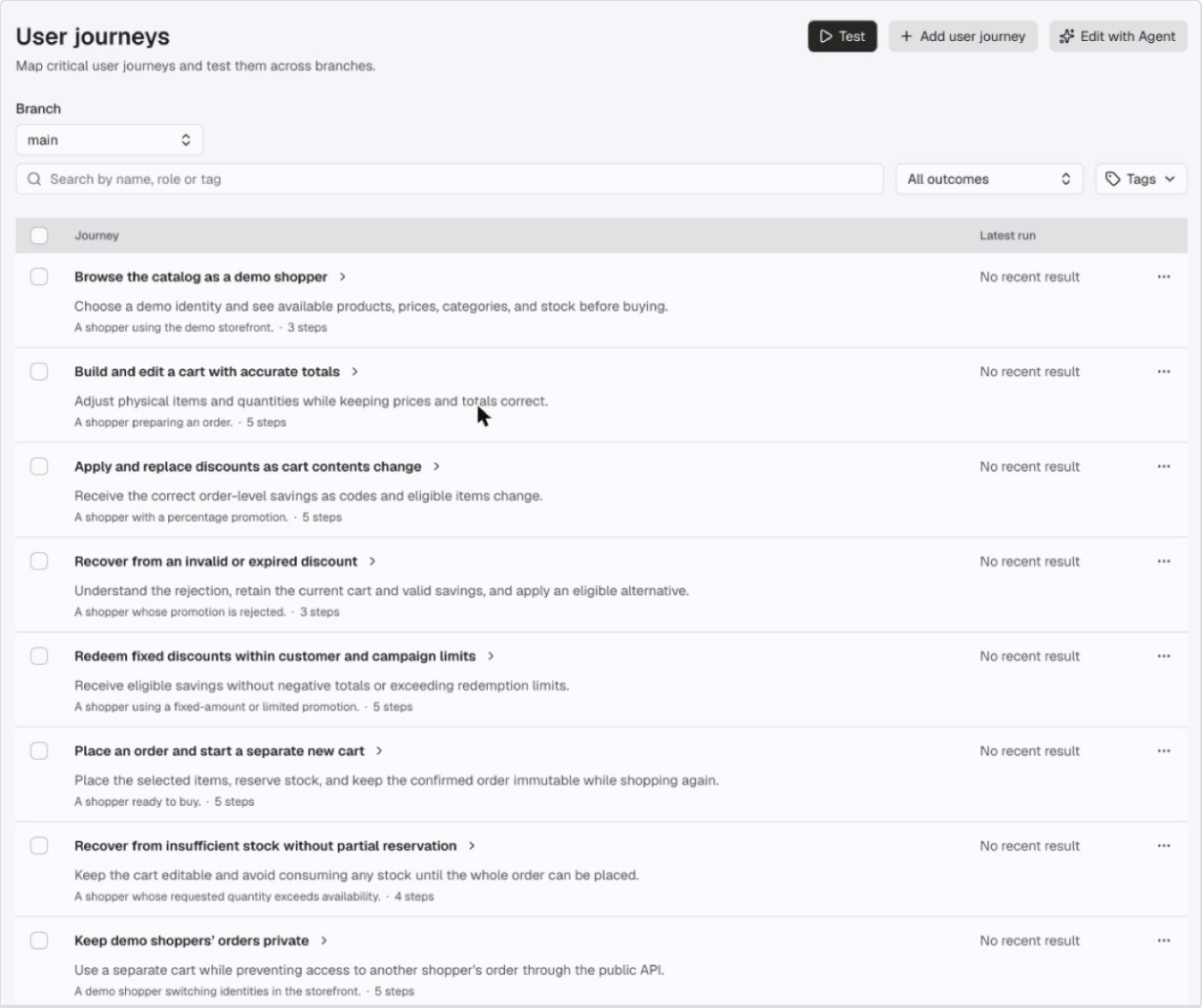
F1 in Deep Scan: CWE, reachability, and evidence path.

Stage 6 · QA user journeys: same tests, two branches

The QA Agent mapped nine critical user journeys from the repository's source and saved them as a reusable library under Testing → Test suites → `mmani-demo-prs`. The same library runs against any branch.

STEP 21 **Map the journey library with an agent** ↑ Index

DO	SAY	SEE
Open Testing → Test suites → `mmani-demo-prs` → User journeys. For a new repo, click **Map with an Agent**, review the proposal, and reply "Apply all nine journeys."	*"PR testing checks one change. Journeys check the product: browsing, carts, discounts, checkout, stock, privacy, and the fulfillment handoff — written once, run on every branch."*	Nine journeys, each with a user, a goal, test steps, and assertions grounded in the source.

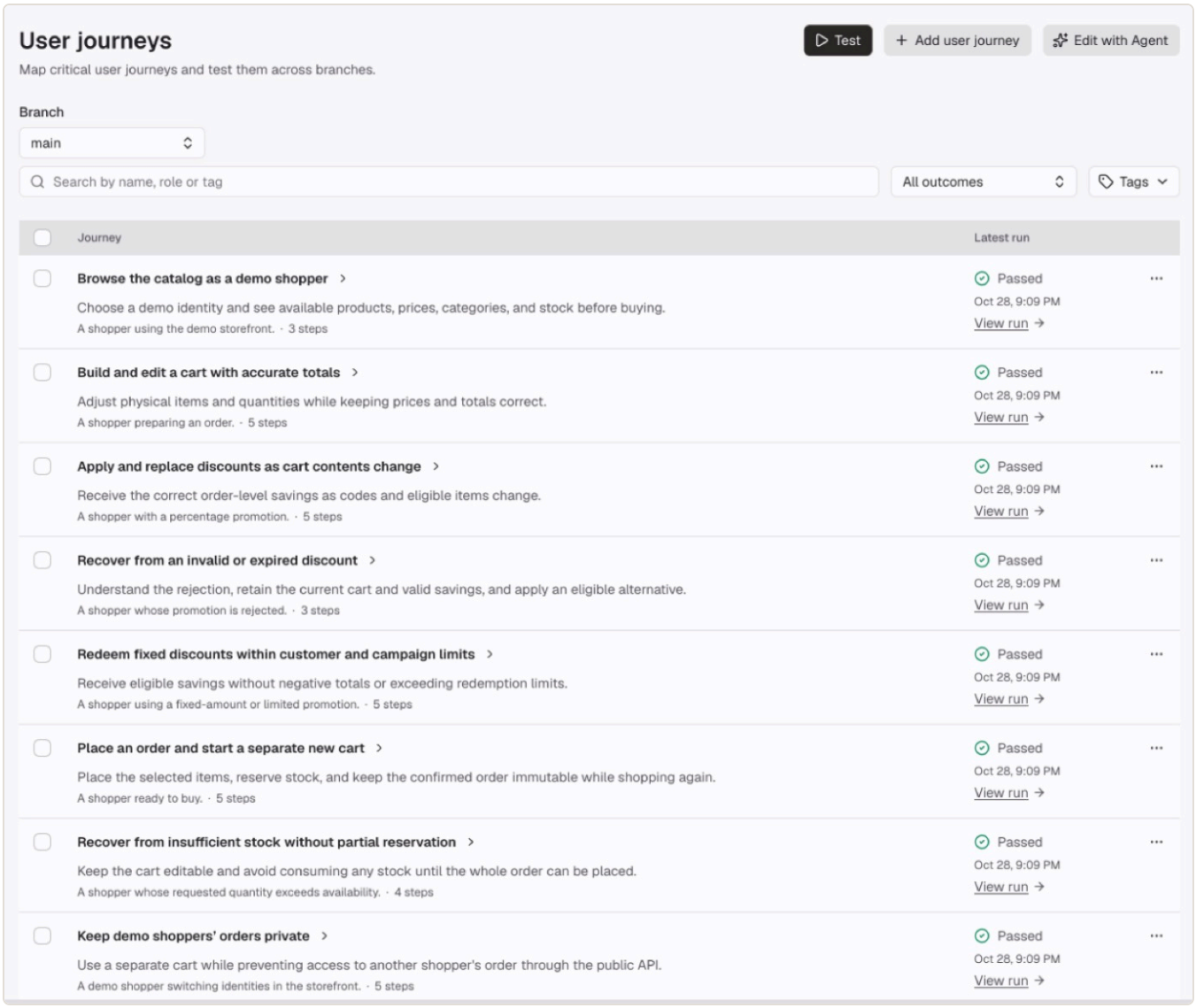


The journey library for mmani-demo-prs.

STEP 22 Baseline: all nine pass on `main`

↑ Index

DO	SAY	SEE
Set Branch to `main`. Show the latest run.	*"On main — the corrected discount code — every journey passes. That is our baseline."*	All nine journeys Passed (run of Oct 28, 9:09 PM, commit `43b4219`).

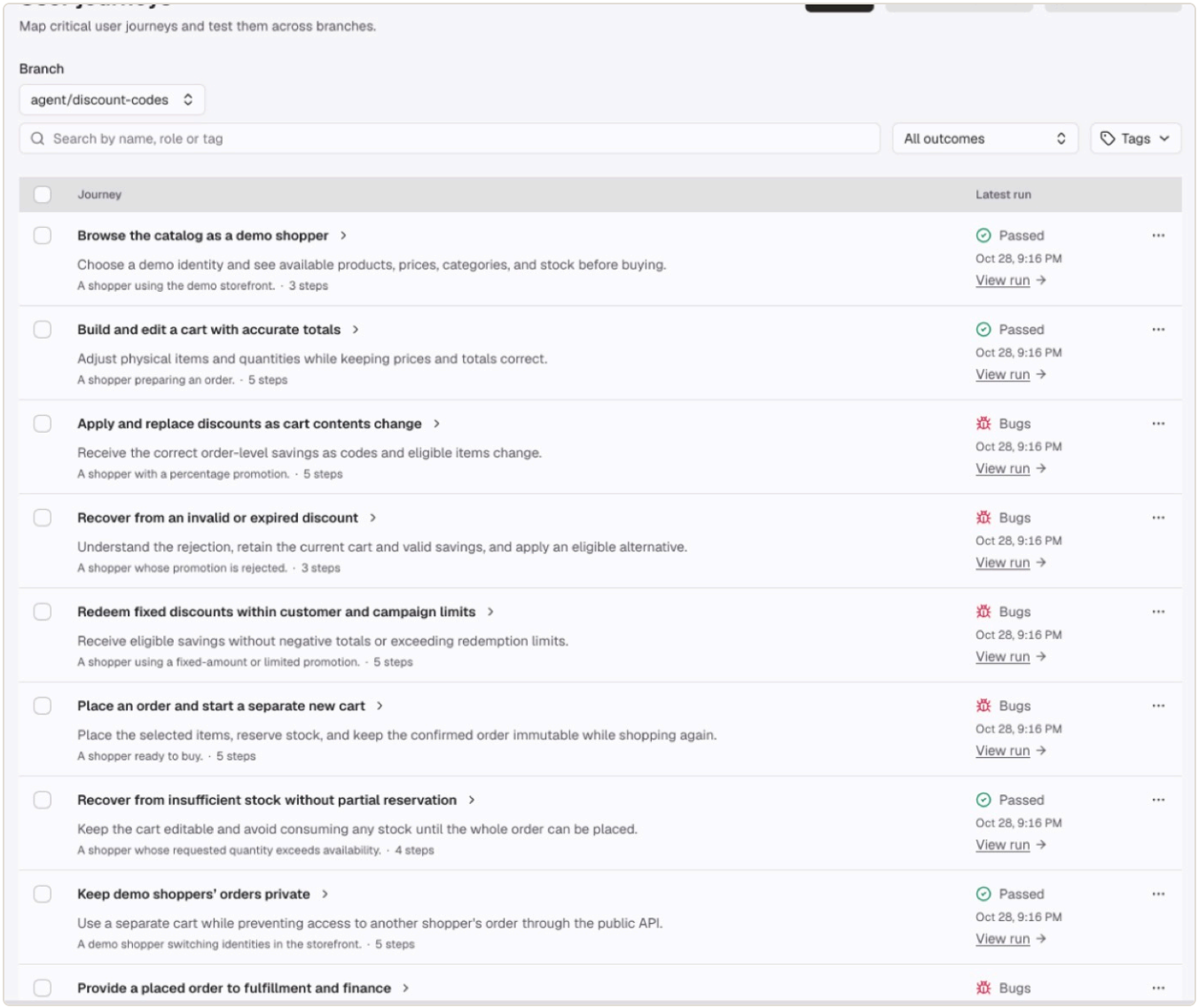


All journeys pass on main.

STEP 23 F2 Same journeys on the agent branch: five fail

↑ Index

DO	SAY	SEE
Set Branch to `agent/discount-codes`. Click **Test**, keep all nine journeys, and run (or show the existing run).	*"Nothing about the tests changed — only the branch. Five journeys now fail, and every failure is about discounts."*	Bugs on: apply and replace discounts; recover from an invalid or expired discount; redeem fixed discounts; place an order; fulfillment and finance handoff. Browsing, cart editing, stock recovery, and privacy still pass.

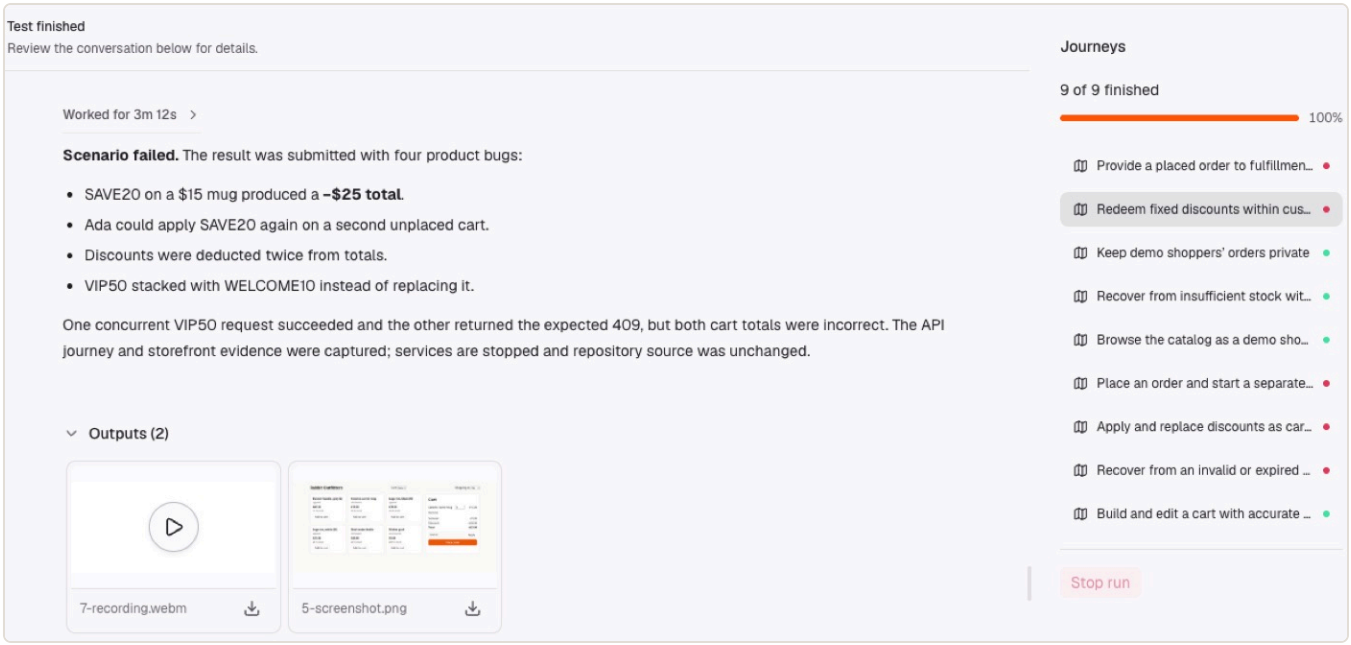


The same nine journeys on agent/discount-codes (run of Oct 28, 9:16 PM).

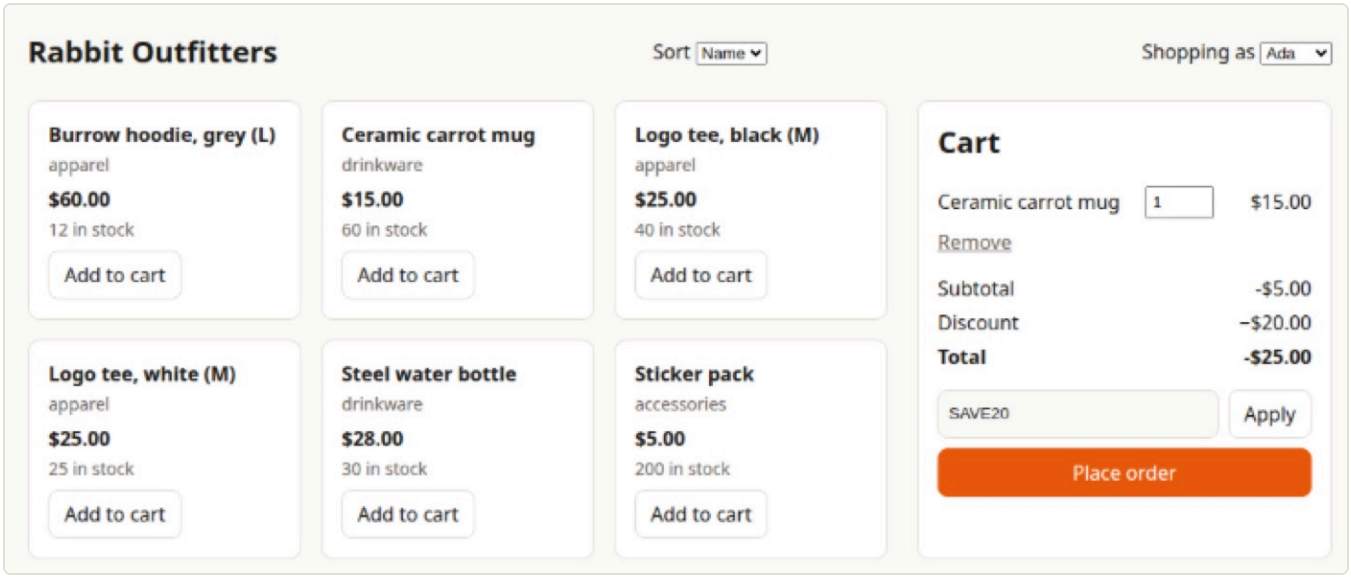
STEP 24 F2 Journey evidence: negative totals and a broken handoff

↑ Index

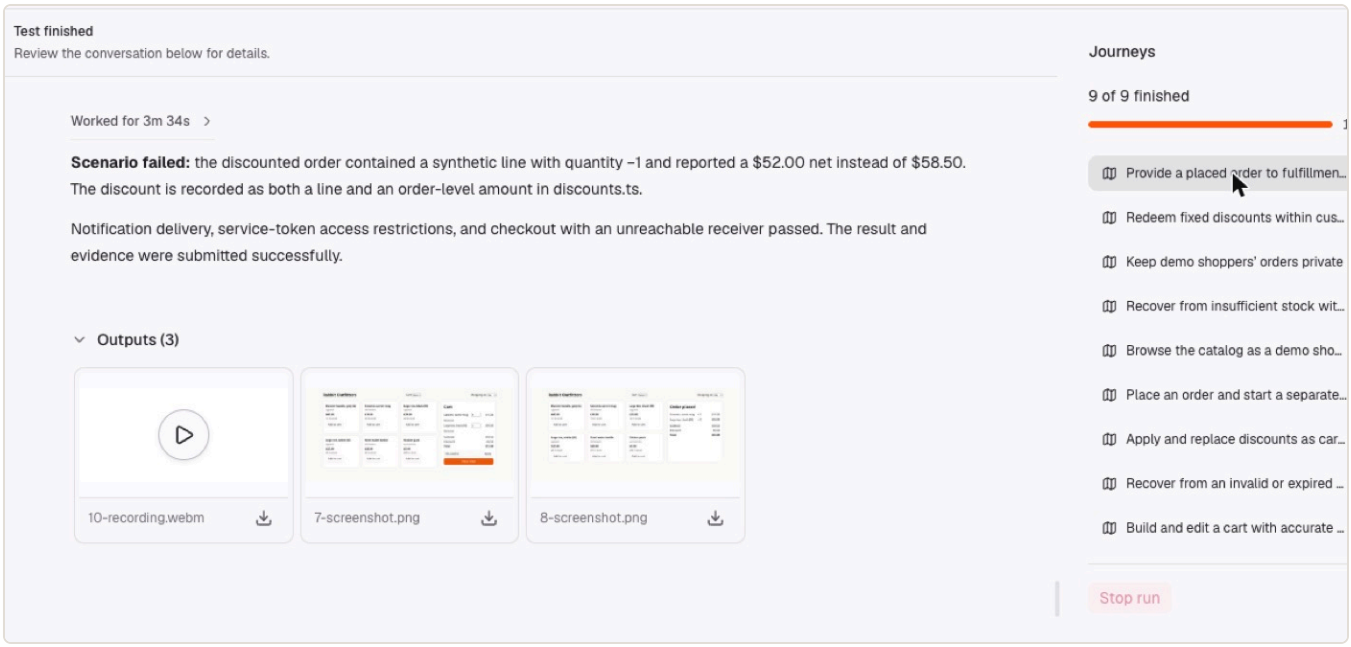
DO	SAY	SEE
Open *Redeem fixed discounts within customer and campaign limits* → View run, and open its storefront screenshot. Then open *Provide a placed order to fulfillment and finance*.	*"Here is what a shopper actually sees: a $15 mug, a $20 code, and a total of minus $25. And the order the warehouse receives has a line with quantity minus one."*	"SAVE20 on a $15 mug produced a –$25 total." "Discounts were deducted twice from totals." Storefront: Subtotal –$5.00, Discount –$20.00, Total –$25.00. Fulfillment: "a synthetic line with quantity –1 … a $52.00 net instead of $58.50."



Fixed-discount journey: four product bugs, with recording and screenshot.



The storefront during the journey: a –$25.00 total.



Fulfillment handoff journey: quantity −1 line and a double-counted net.

WOW One journey library, written once from the source. On main it is green; on the agent branch it reproduces F2 from the shopper's screen to the warehouse's order feed.

THE LINE THAT LANDS "One ticket, one agent, two mistakes — and every stop saw them: my laptop, the PR, the reviewer's queue, the architecture view, the test run, and the security scan. Nothing reached main."

Part III — Reference

The finding trail

STOP	F1 · SQL INJECTION	F2 · DISCOUNT TOTALS
CLI	CWE-89, `discounts.ts:20`	Through the tests: `discounts.test.ts:26`, `:52–54`
VS Code	Inline at `discounts.ts:20`	Ln 59 (clamp), Ln 67-72 (order line)
PR review	CWE-89, Major	:73, :30, db.ts:17
Pre-merge checks	Security Gate error; S7 Not met	Order Invariants error; S1–S3 Not met
Triage	PR #3 card: Medium risk, High blast radius	
Change Stack	Layer 2; Blast Radius; architecture impact	Layer 3; architecture impact (retained concern)
QA Agent	Scenario 3: injected code returned 200 and applied WELCOME10	Scenario 2: subtotal 4,000 vs 6,000; mug net –2,500; consumer line quantity –1
User journeys	—	5 of 9 journeys fail on the agent branch (all pass on main): –$25.00 storefront total; quantity –1 in the fulfillment feed
Deep Scan	High, CWE-89, `discounts.ts:20`, reachable from the gateway route	Medium, CWE-682, `discounts.ts:66`

Objection handling

OBJECTION	RESPONSE
"Our tests pass, so it's fine."	The agent's tests assert –$25.00 and $52.00. Show the Success Criteria check: it requires tests that assert the values in the ticket.
"We already run SAST."	Keep it. F2 is not a SAST finding: it needs the ticket, the org's money rules, and the consumer repo to recognize.
"Does the check block the merge?"	The checks report errors. Branch protection must separately require them.
"Is the consumer actually deployed against this app?"	No. The verified moment is CodeRabbit reading the consumer's code and naming the broken call sites.

Freshness check before every demo

1. PR #3 is open, unmerged, labeled `Demo – Do not merge`; F1 and F2 threads are unresolved.
2. The PR summary still shows Pre-merge checks with Success Criteria graded against SC-10.
3. The Blast Radius link on the F1 comment opens the graph (use the comment's link; the bare Change Stack tab may say it was not generated for the latest run).
4. Locally: branch `agent/discount-codes` and base `demo/agent-base` exist.
5. Do not push to `agent/discount-codes`; every push triggers a new review.
6. Deep Scan: show scan `2b10cbc0` (agent branch). A scan of `main` does not contain F1.

Appendix A — Deep-link index

MOMENT	LINK
Jira story	SC-10
Pull request	PR #3
F1 on the PR	discussion_r4407733137
F2 on the PR	discussion_r4407733154, …140, …132
Triage	app.coderabbit.ai/triage?repo=mmani-demo-prs
Change Stack	overview · layer 2
QA Agent run	PR #3 QA run
Deep Scan	scan 2b10cbc0 findings

Appendix B — Focused PRs for single-feature demos

PR	FEATURE	JIRA
#4 Order history · #11 docstrings	Walkthrough; Chat (generate docstrings)	SC-11
#5 Keep fixed discounts	Functional bug (negative net)	SC-13
#6 Drop write lock	Data integrity (non-atomic redemption)	SC-14
#7 Receipt lines	Cross-repo breaking change	SC-15
#8 Low-stock badge	Issue validation; out of scope	SC-17

Appendix C — Local setup for the CLI and VS Code demos

Steps 3 and 4 review the coding agent's branch on your laptop. The branch must be checked out, and the review must compare it against the commit the agent started from — not `main`, which now holds the corrected reference implementation. Run every command from a terminal; nothing here pushes or changes the shared repository.

1. Prerequisites

- Git and GitHub access to `coderabbit-demo/mmani-demo-prs` (public).
- CodeRabbit CLI installed (see docs.coderabbit.ai/cli) and VS Code with the CodeRabbit extension.
- Signed in to the **coderabbit-demo** organization in both.

```
coderabbit auth login
coderabbit auth status        # Account and organization
coderabbit auth org           # switch to coderabbit-demo if needed
coderabbit doctor             # checks the CLI is ready for local reviews
code --install-extension coderabbit.coderabbit-vscode
```

2. Clone the repository and check out the agent's branch

```
cd ~/projects
git clone https://github.com/coderabbit-demo/mmani-demo-prs.git
cd mmani-demo-prs
git switch agent/discount-codes
git log --oneline -1          # expect: e7a13a2 Merge branch 'main' into agent/discount-codes
```

Already cloned? Run `git fetch origin` and then `git switch agent/discount-codes && git pull --ff-only`.

3. Create the comparison base

The base is the commit where the agent's branch and `main` last met. Create a local branch for it so both tools can select it by name.

```
git branch -f demo/agent-base "$(git merge-base origin/agent/discount-codes origin/main)"
git rev-parse --short demo/agent-base               # expect: df75131
git diff --stat demo/agent-base..HEAD | tail -1     # expect: 13 files changed, 251
insertions(+), 13 deletions(-)
```

`demo/agent-base` is local only. Never push it.

4. Run the CLI review (Step 3)

```
coderabbit review --committed --base demo/agent-base
coderabbit review findings    # list the stored findings again without a new review
```

Expect F1 at `services/orders/src/discounts.ts:20` and F2 at `discounts.ts:59` and in `tests/discounts.test.ts`. Exact wording varies between runs.

5. Run the VS Code review (Step 4)

```
code ~/projects/mmani-demo-prs
```

1. Open the CodeRabbit panel in the activity bar.
2. Check **Current** shows `agent/discount-codes`.
3. Click the pencil next to **Compare against** and choose `demo/agent-base`. Do not leave it on `origin/main`.
4. Set **Changes to review** to **Committed changes** (**All changes** is equivalent while the working tree is clean).
5. Click **Review**. Expect about 18 findings, with F1 inline at line 20.

6. Reset after a rehearsal

If you applied a suggestion or used Fix with AI locally, discard it before the next demo. This only affects your local checkout.

```
git status --short                          # see what changed
git restore --staged --worktree .           # discard edits to tracked files
git switch agent/discount-codes
git reset --hard origin/agent/discount-codes
```

Do not push from this checkout. Any push to `agent/discount-codes` triggers a new PR review on PR #3.